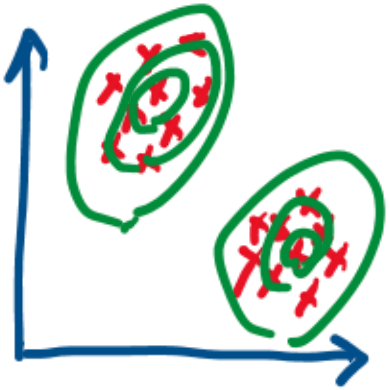
Deep (Vision) Generative Models

Generative modeling

- Learns P(X,Y) or P(X|Y)
  - Can sample (generates) X from P(X|Y)
  - Y is the controlling parameter

- What is P(X|Y)?
  - Most of the time it's assumed to be a parametric distribution



Generating real world data

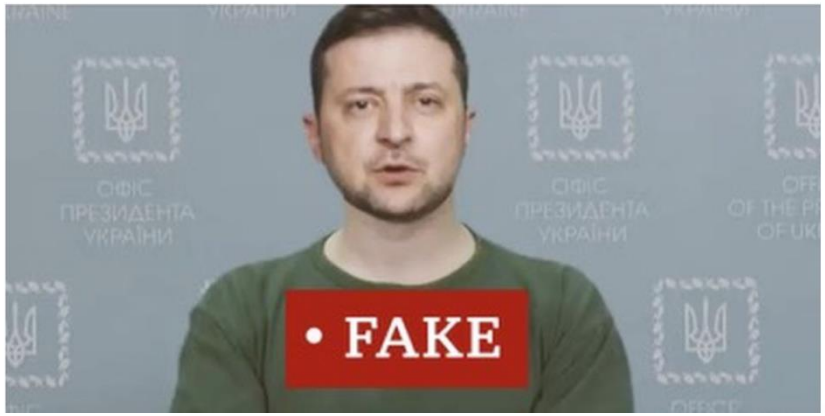
- Let’s say we want to generate faces
  - How do we fit a distribution to it?

Deepfake presidents used in Russia-Ukraine war

By Jane Wakefield
BBC Technology

5 days ago

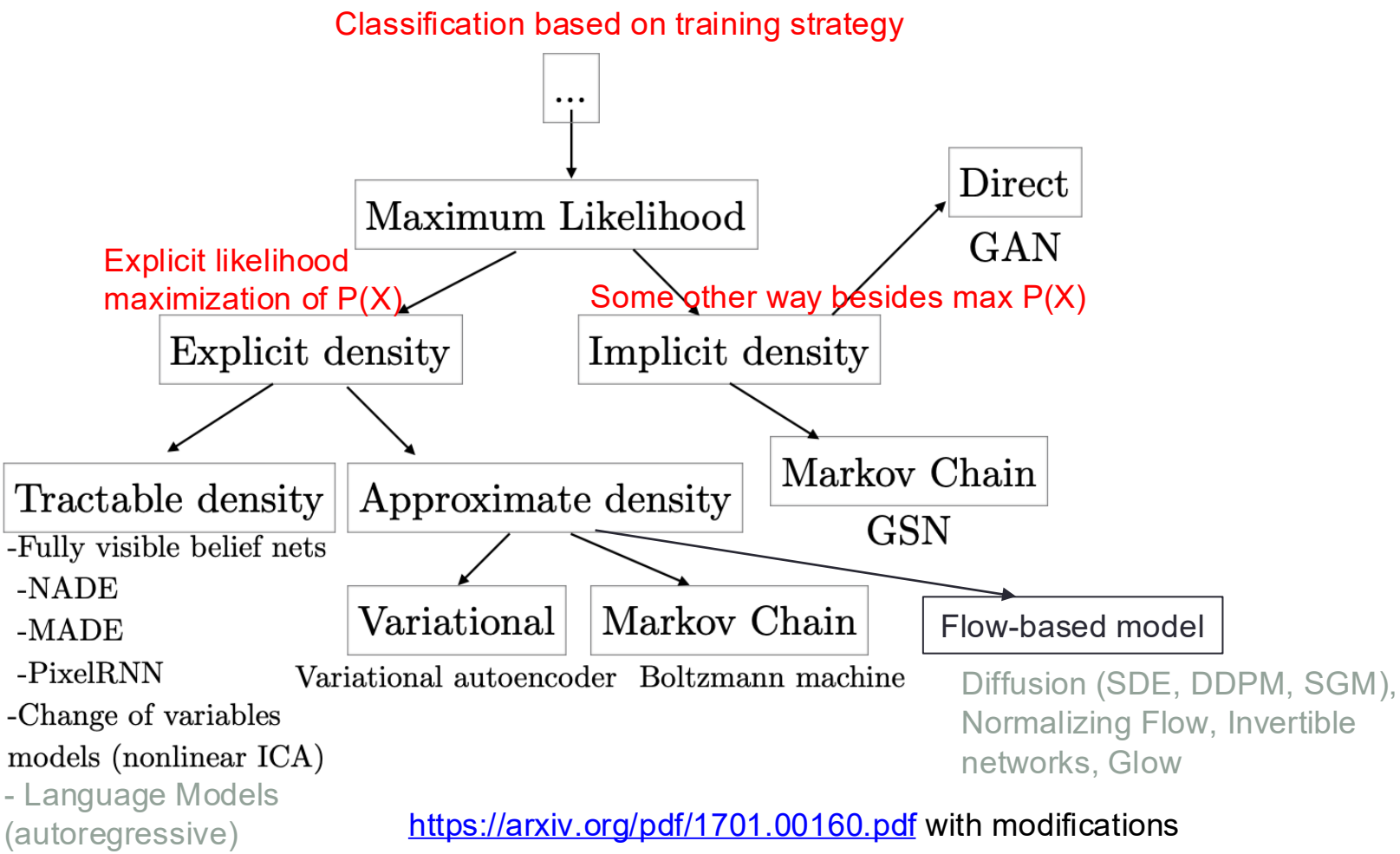
Russia-Ukraine war



https://www.bbc.com/news/technology-60780142

Implicit generative models

- We can generate without explicitly modeling P(X)



https://arxiv.org/pdf/1701.00160.pdf with modifications

Another view

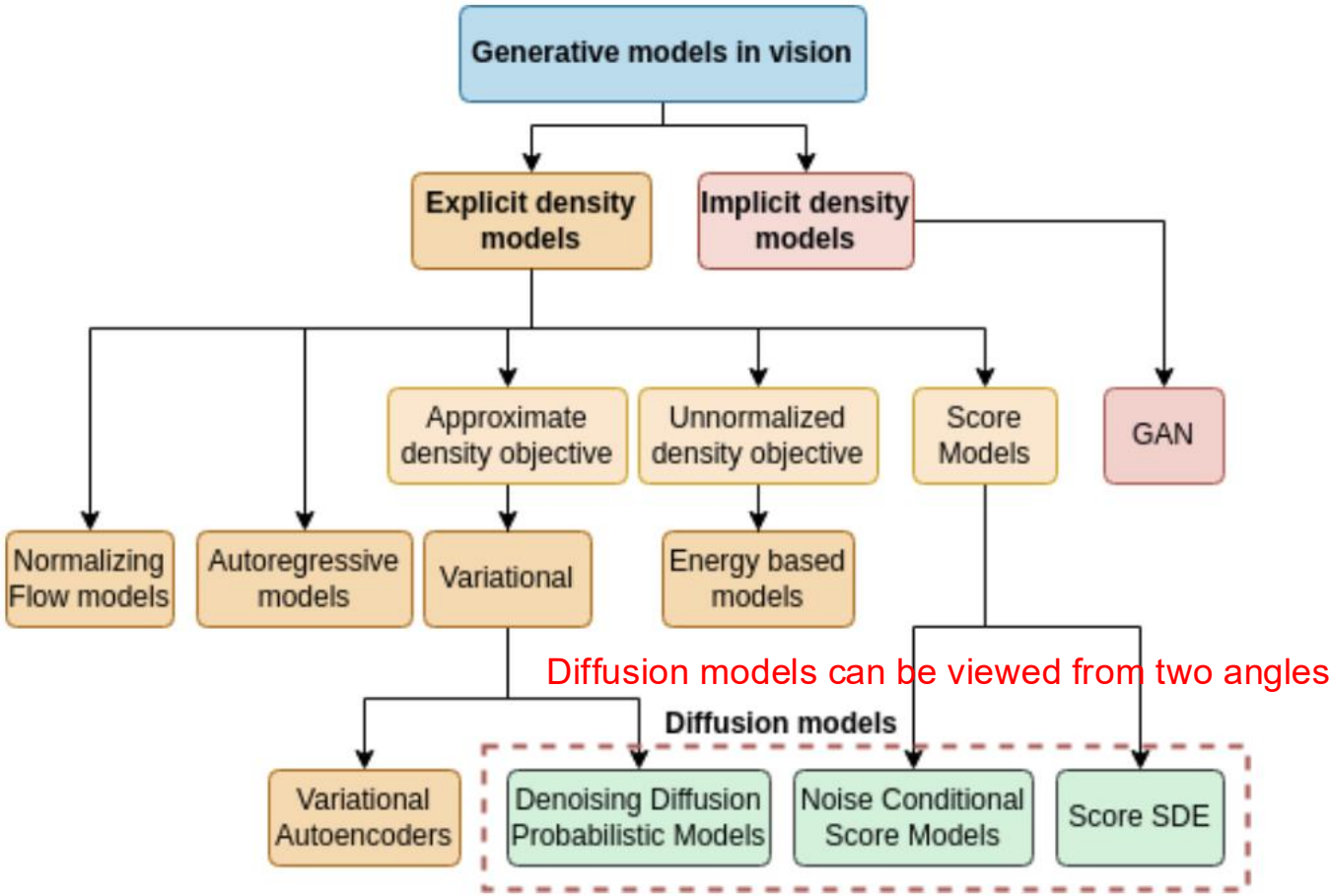


Figure 2. An extension of generative models classification based on[25]

https://arxiv.org/pdf/2402.16369v1

Deep generative models

- Autoencoders
  - Variational autoencoders
- GAN
- Diffusion/Flow

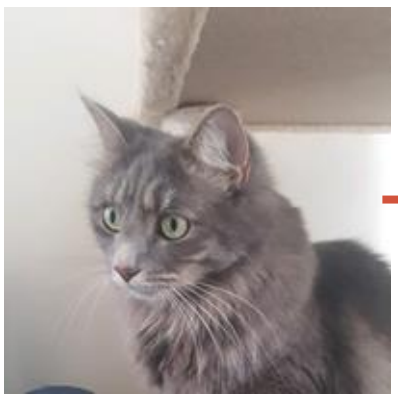
DNN as a representation learning model

We know that DNN can model highly complex mapping.

→ Deep Neural Network → (4.1, 5.0)

Autoencoder

Auto means self!



Input

X

Encoder → (?, ?) → Decoder →

Output

X'

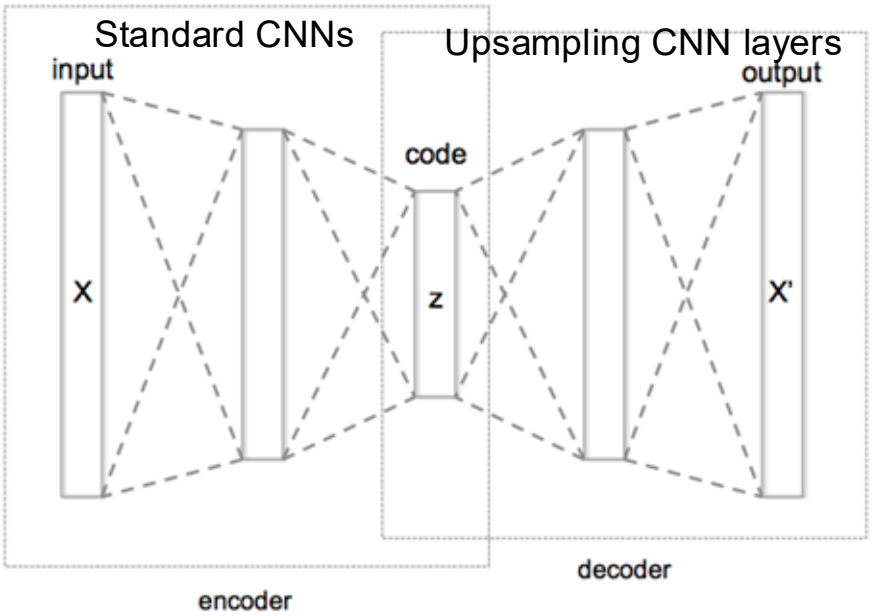
Now we can train the network with L2 Loss: $\|X - X'\|^2$

Autoencoder

Auto means self!

Input

X

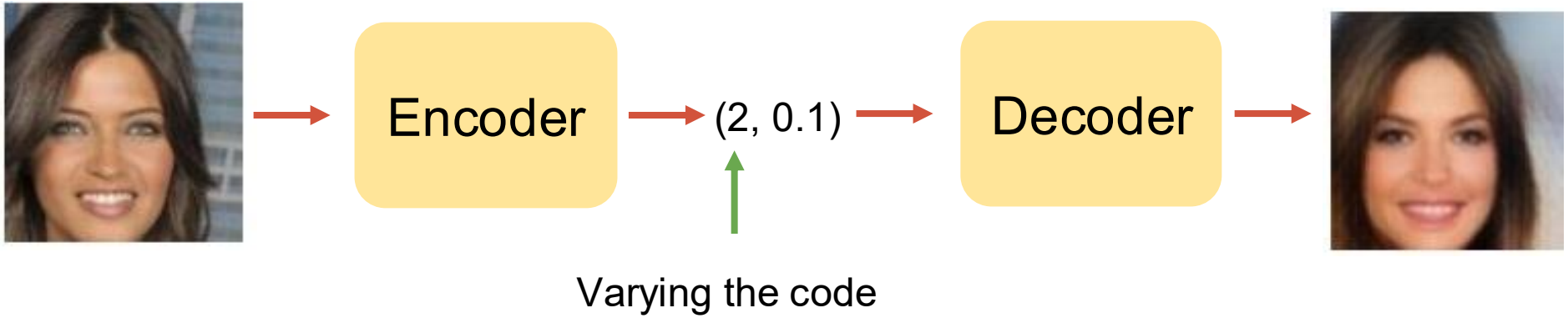


Output

X'

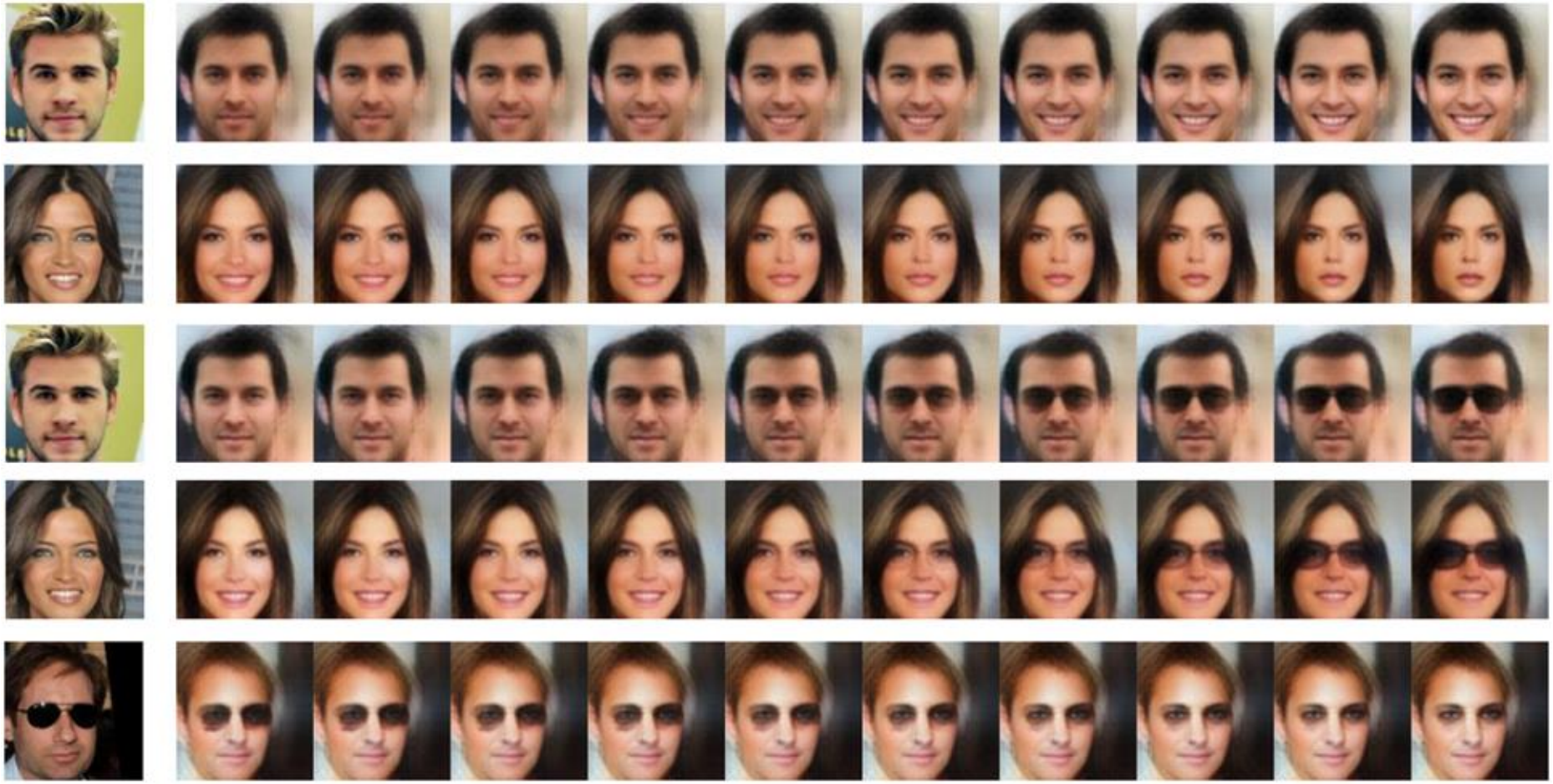
Now we can train the network with L2 Loss: $\|X - X'\|^2$

Inspecting attributes learned from AE



Deep Feature Consistent Variational Autoencoder [Hou et al. 2016]

Attributes from autoencoder

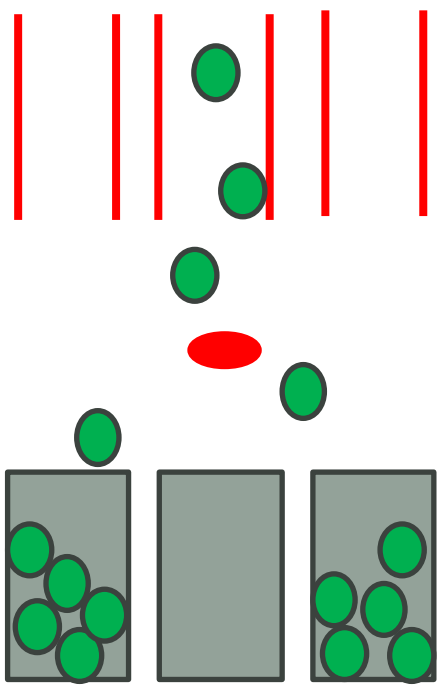


Deep Feature Consistent Variational Autoencoder [Hou et al. 2016]

Deterministic mapping

Autoencoder is good, but this learns a deterministic mapping from X $\xrightarrow[\text{encode}]{}$ Z $\xrightarrow[\text{decode}]{}$ X' (point estimate)

Let's say the loss function is L2 $\|X - X'\|^2$



A deterministic mapping can lead to improbable answers. We need some non-deterministic mapping (distribution learning)

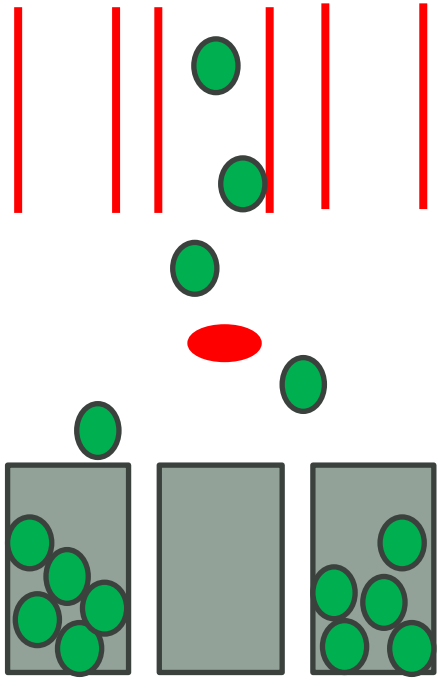
Distribution learning

One simple way to think of deep distribution learning is we're learning a mapping to a distribution instead

$[X_1,\ldots, X_n]$ -> distribution -> $[X'_1,\ldots, X'_m]$

distribution fitting

sampling



Variational Autoencoder

Original and by-product motivations:

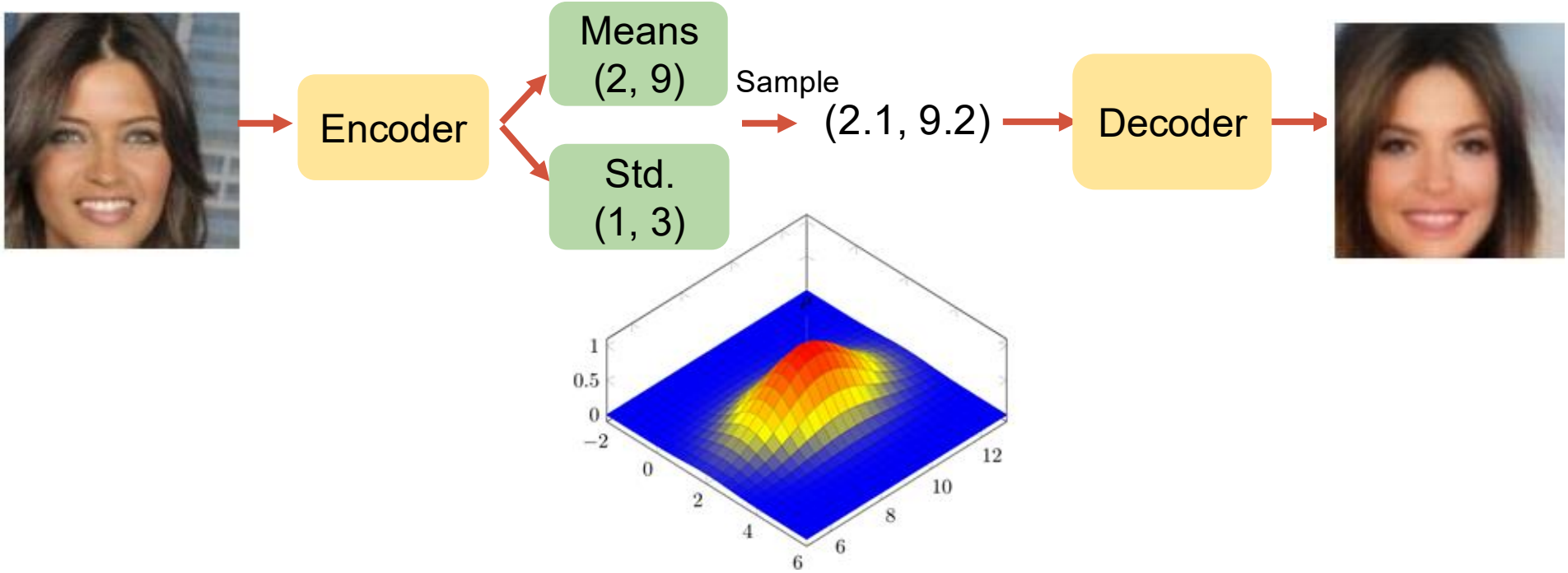
0. Learning a non-deterministic mapping

1. Allows approximation of intractable posterior.

2. Generate photos from the decoder

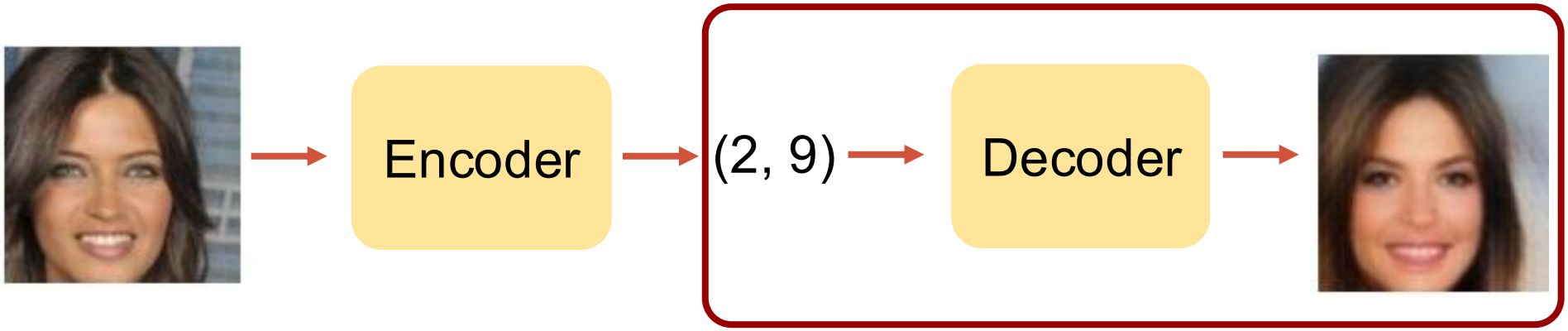
Motivation #0 – learning a non-deterministic mapping

Probabilistic AE



The encoder learns the parameters of a distribution instead
Then, we sample to generate.

Motivation #2 -- Generating Photos



Use this part to generate an image from random code.

But how do you generate the two numbers? Uniform? Gaussian? Encoder could output two numbers with some crazy distribution.

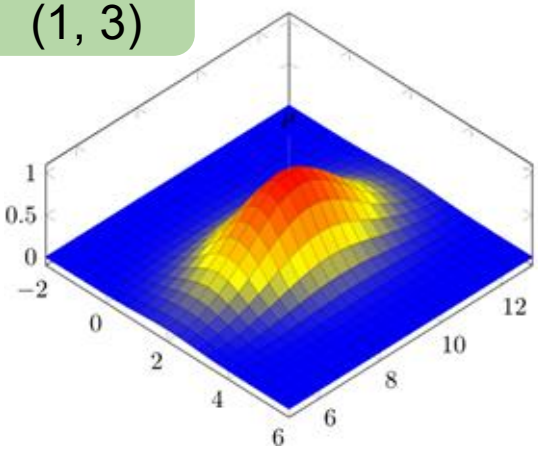
VAE allows us to force the latent code to follow some distribution like a unit normal distribution.

Standard AE

Encoder → (2, 9) → Decoder →

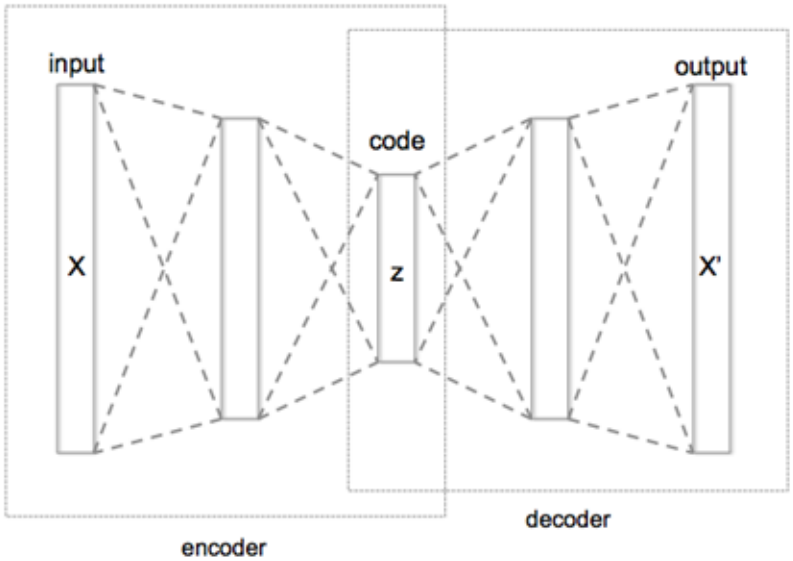
Probabilistic AE

Encoder → Means (2, 9), Std. (1, 3) → Sample → (2.1, 9.2) → Decoder →



Motivation #1 -- Solving Intractable Inference

Autoencoder can be viewed from a probabilistic perspective.



However, a standard AE isn't quite "probabilistic."

I.e., the encoder deterministically outputs z given x. Rather we want a distribution of the code (posterior p(z | x)).

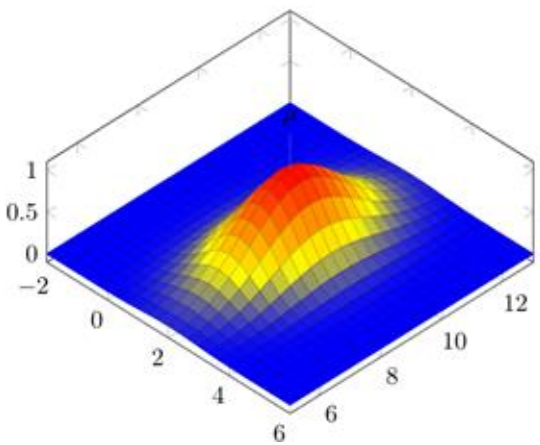
Autoencoding Variational Bayes: Bayesian Review

One important inference problem : Posterior inference

$$p(\mathbf{z} \mid \mathbf{x})$$

$\mathbf{x}$ Observed data

$\mathbf{z}$ Latent variable (hidden code)



Bayesian Review

One important inference problem : Posterior inference

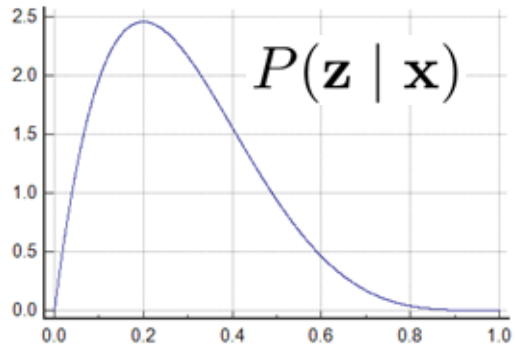
$$p(\mathbf{z} \mid \mathbf{x})$$

$\mathbf{x}$ Observed data

2 heads out of 10 coin flips
(Binomial Dist)

$\mathbf{z}$ Latent representation
(hidden code)

The coin fairness
z = 0.5, fair coin
z = 1, always head
z = 0, always tail



Bayesian Review

One important inference problem : Posterior inference

$$p(\mathbf{z} \mid \mathbf{x})$$

$\mathbf{x}$ Observed data

$\mathbf{z}$ Latent representation (hidden code)

Class label cat, color, can fly?, ...

$p(\mathbf{z} \mid \mathbf{x})$

Tells you **not only** what animal is the most likely, **but also** how “orange cat” it is, how “white dog” it is, etc -- the probability of all hidden code

Variational Bayesian Review

One important inference problem : Posterior inference

$$p(\mathbf{z} \mid \mathbf{x}) = \frac{p(\mathbf{z}, \mathbf{x})}{p(\mathbf{x})} = \frac{p(\mathbf{z}, \mathbf{x})}{\boxed{\int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}}}$$

Hard to compute because z can be high-dimensional

E.g. $\mathbf{z} = \{z_1, z_2, z_3, \ldots\}$

z_1 : class label {cat, dog, rabbit, rat}

z_2 : color, a real number

z_3 : ability to fly {yes, no}

$$\int d\mathbf{z} = \sum_{z_1} \int_{z_2} \sum_{z_3} \ldots dz_2$$

Bayesian Review

One important inference problem : Posterior inference

$$p(\mathbf{z} \mid \mathbf{x}) = \frac{p(\mathbf{z}, \mathbf{x})}{p(\mathbf{x})} = \frac{p(\mathbf{z}, \mathbf{x})}{\boxed{\int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}}}$$

A solution:

Let's approximate the true posterior $p(\mathbf{z} \mid \mathbf{x})$

with an approximate, easier-to-work-with $q(\mathbf{z} \mid \mathbf{x})$

and solve this as an optimization problem.

Approximate True Posterior

Let's approximate the true posterior $p(\mathbf{z} \mid \mathbf{x})$ with an approximate, easier-to-work-with $q(\mathbf{z} \mid \mathbf{x})$ and solve this as an optimization problem.

Approximate True Posterior

Let's approximate the true posterior $p(\mathbf{z} \mid \mathbf{x})$ with an approximate, easier-to-work-with $q(\mathbf{z} \mid \mathbf{x})$ and solve this as an optimization problem.

Best $q^*(\mathbf{z} \mid \mathbf{x})$ is the one closest to $p(\mathbf{z} \mid \mathbf{x})$

A solution: $q^*(\mathbf{z} \mid \mathbf{x}) = \underset{q}{\operatorname{argmin}} \, \mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right]$

- But why not $\mathrm{KL}\left[p(\mathbf{z} \mid \mathbf{x}) \| q(\mathbf{z} \mid \mathbf{x})\right]$ or $D\left(p(\mathbf{z} \mid \mathbf{x}), q(\mathbf{z} \mid \mathbf{x})\right)$?
- How is this related to autoencoder?

Approximate True Posterior

Let's approximate the true posterior $p(\mathbf{z} \mid \mathbf{x})$ with an approximate, easier-to-work-with $q(\mathbf{z} \mid \mathbf{x})$ and solve this as an optimization problem.

Because doing this (i.e., using KL (q || p)):

$$q^*(\mathbf{z} \mid \mathbf{x}) = \underset{q}{\operatorname{argmin}} \operatorname{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right]$$

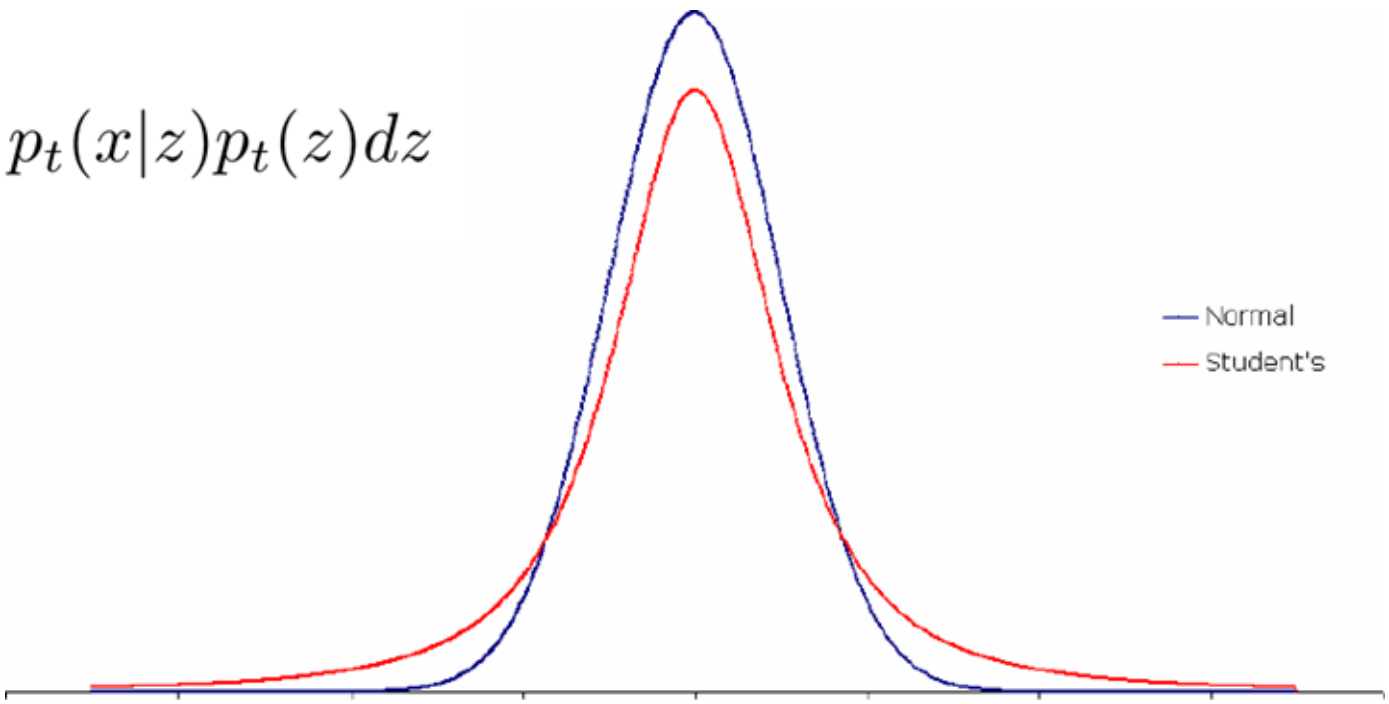
Corresponds to maximizing the evidence lower bound (ELBO)!

Digression: Model evidence

Model evidence (or marginal likelihood) is simply $p(\mathbf{x})$

$$p_g(x) = \int p_g(x|z)p_g(z)dz$$

$$p_t(x) = \int p_t(x|z)p_t(z)dz$$



$P(X)$ – evidence

Recall EM algorithm

- Goal of EM : $\log \sum_k p(x, k;\theta) >= \overbrace{\Sigma_k Q(k) \log (p(x, k;\theta)/Q(k))}^{\text{ELBO}}$
- Maximize the ELBO instead
- Initialize Θ
- Repeat till convergence
  - Expectation step (E-step) : estimate the conditional expectation $Q(k) = p(k|x;\theta)$ using the current θ.
  - Maximization step (M-step) : Estimate new Θ given by maximizing the ELBO given current Q(k)

$$E_{k\sim Q}\left[\log \frac{P(x,k)}{Q(k)}\right]$$

$P(X|Y)$ – likelihood

$P(Y|X)$ – posterior

Proof

This KL is hard to compute

$$q^*(\mathbf{z} \mid \mathbf{x}) = \underset{q}{\operatorname{argmin}}\, \mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right]$$

$$\mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right] = \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x})] - \mathbb{E}_q[\log p(\mathbf{z} \mid \mathbf{x})]$$

$q(z|x)$

(by definition)

$$= \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x})] - \mathbb{E}_q[\log p(\mathbf{z}, \mathbf{x})] + \log p(\mathbf{x})$$

(expanding P(z | x), E_q[log p(x)] = log p(x))

$$= \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})] + \log p(\mathbf{x})$$

Constant w.r.t q. So, we can safely kill it

$$\min_q \mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right] \iff \min_q \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})]$$

Questions: 1. What's this new term? 2. Is it easier to evaluate?

Proof

$$\min_q \mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right] \iff \min_q \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})]$$

Let's interpret this new term:

From previous slide, we have:

$$\log p(\mathbf{x}) = \mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right] - \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})]$$

We know KL >= 0, always!

$$\log p(\mathbf{x}) \geq \boxed{-\mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})]}$$

(Lower bound of model evidence)

$$\begin{aligned}\min_q \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})] &\iff \max_q -\mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})] \\ &\iff \max_q \mathrm{ELBO}\end{aligned}$$

But how to compute this ELBO term then?

Maximizing ELBO

$$q^*(\mathbf{z} \mid \mathbf{x}) = \operatorname*{argmin}_q \mathrm{KL}\left[q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right]$$

$$\max_q \mathrm{ELBO} = \max_q -\mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})]$$

$$\iff \min_q \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z}, \mathbf{x})]$$

$$\iff \min_q \mathbb{E}_q[\log q(\mathbf{z} \mid \mathbf{x})] - \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{z})]$$

$$\iff \min_q \mathrm{KL}[q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$$

We're free to pick

Standard AE

Encoder → (2, 9) → Decoder →

Variational AE

$q(z|x)$

Encoder → Means / Std.

Sample → (2.1, 9.2)

$q(\mathbf{z} \mid \mathbf{x})$

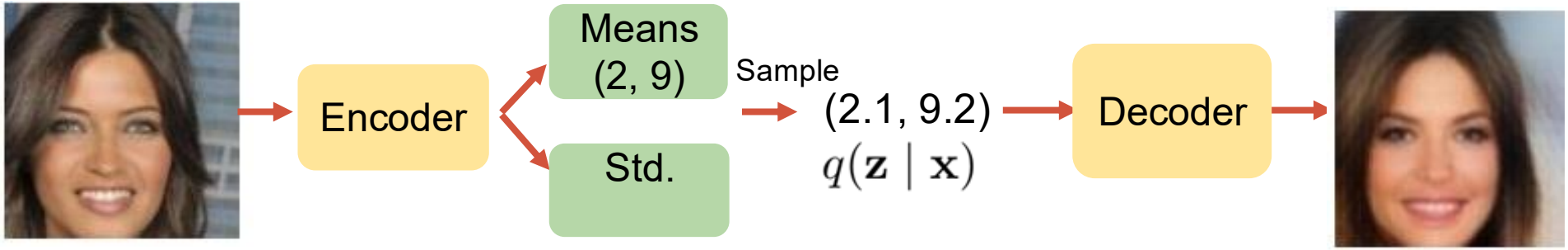
$p(x|z)$

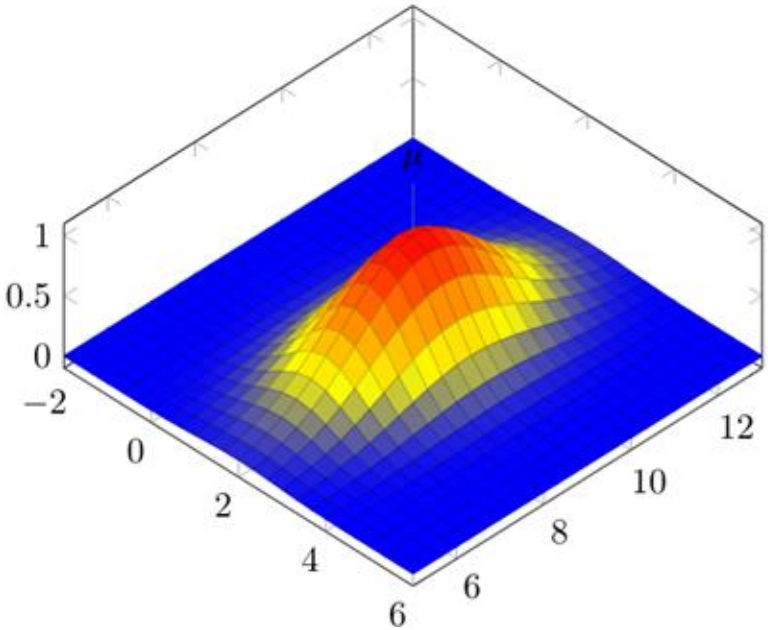
Decoder →

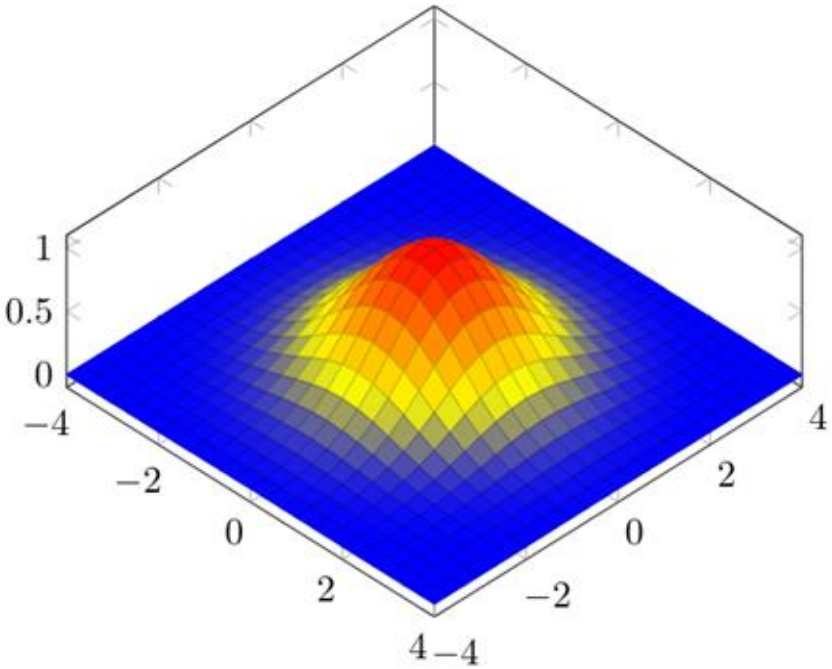
*Output as a sample from P(x | z).

$$\min_q \mathrm{KL}[q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$$

sample from $q(z|x)$







$q(\mathbf{z} \mid \mathbf{x})$ is forced to be close to $p(\mathbf{z}) = \mathcal{N}(0, I)$

$$\min_q \mathrm{KL}[q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$$

What is this second term?

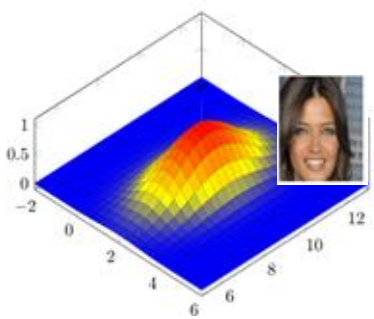
Second Term: Data Log Likelihood

$$\min_q \mathrm{KL}[q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$$

If we also model $p(\mathbf{x} \mid \mathbf{z})$ as a Gaussian distribution, where the mean = u is the output of the decoder:

$$p(\mathbf{x} \mid \mathbf{z}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-u)^2}{2\sigma^2}}$$

(Let's consider a single pixel case
u is the output from the decoder
x is the ground-truth pixel)



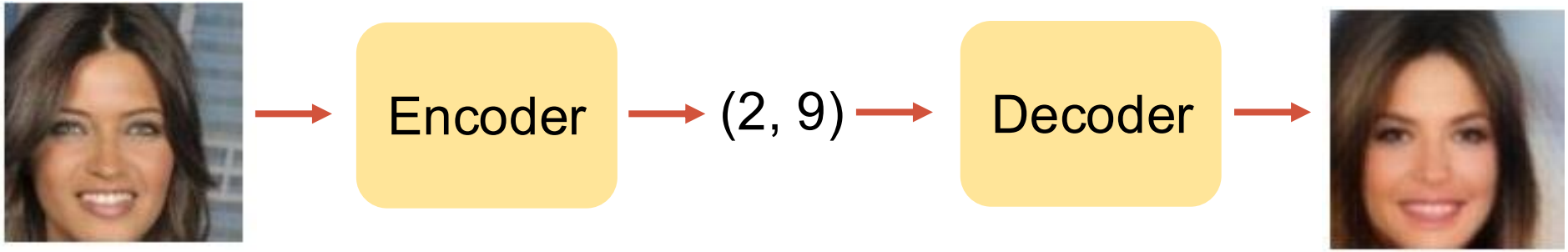
$$\propto \exp\left(-(x-u)^2\right)$$

$$-\log[\exp\left(-(x-u)^2\right)] = (x-u)^2$$

(with constants hidden)

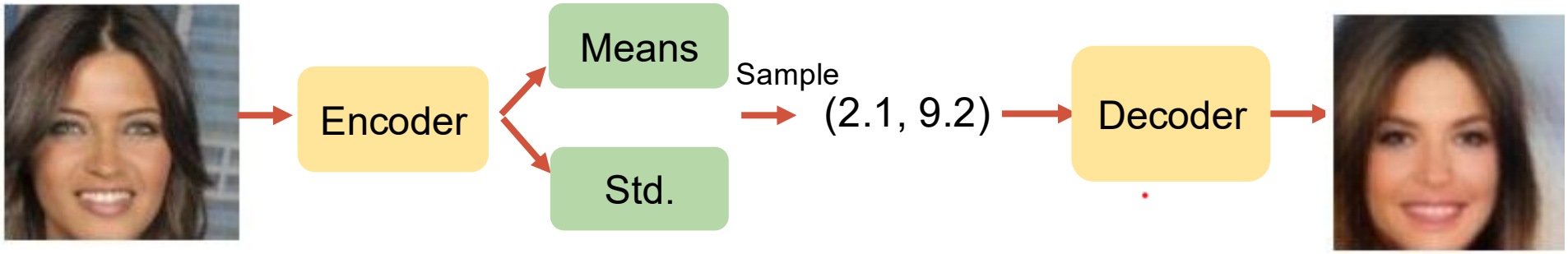
Becomes the standard L2 error

Yet another reason why assuming Gaussian is nice



X X'

Standard AE Loss $\|X - X'\|^2$



X X'

Regularization of L2 loss

Variational AE Loss $\approx \|X - X'\|^2 + \mathrm{KL}[q(\mathbf{z} \mid \mathbf{x}) \| \mathcal{N}(0, I)]$

Precisely, $\min_{q_\phi(\mathbf{z}|\mathbf{x})} \mathrm{KL}[q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]$

Implementation of KL term

$$\min_{q_\phi(\mathbf{z}|\mathbf{x})} \mathrm{KL}[q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]$$

approximate via minibatch

- You can appromate the KL term by computing in the minibatch
- If you assume both q and p are Gaussians (vanilla VAE) the equation simplifies
  - https://mr-easy.github.io/2020-04-16-kl-divergence-between-2-gaussian-distributions/

When q is $\mathcal{N}(0,\ I)$, we get,

Note the notation difference (p and q)

$$D_{KL}(p||q) = \frac{1}{2}\left[\boldsymbol{\mu}_p^T \boldsymbol{\mu}_p + tr\left\{\Sigma_p\right\} - k - \log|\Sigma_p|\right]$$

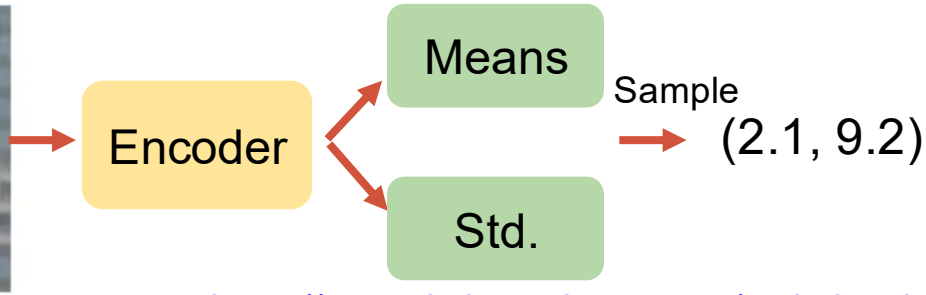
Implementation of KL term

$$\min_{q_\phi(\mathbf{z}|\mathbf{x})} \mathrm{KL}[q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]$$

approximate via minibatch

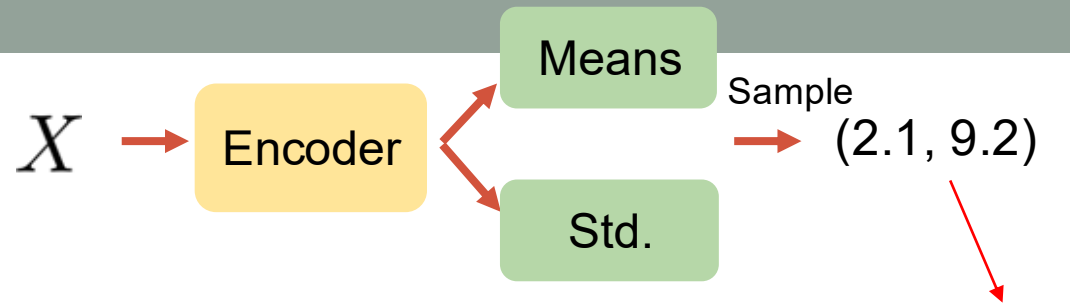
- General form, we have to approximate KL via sampling
- $KL[q|p] = E_{z\sim q(z|x)}[\log q(z|x) - \log p(z)]$
- For image x_i in minibatch
  - Compute $q(z|x)$ by passing through encoder network
  - Sample $z_i \sim q(z|x)$
  - Acummulate $[\log q(z_i|x) - \log p(z_i)]$

X



https://towardsdatascience.com/variational-autoencoder-demystified-with-pytorch-implementation-3a06bee395ed example code

Last Problem



$$\min_{q_\phi(\mathbf{z}|\mathbf{x})} \mathrm{KL}[q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]$$

Estimate in minibatch

When we need to compute the gradient with respect to the network parameters (I.e., ϕ, θ) for SGD, we're struck because we can't backprop through the sampling for computing $\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}$

Example:

$$z = y^2 \qquad z \sim \mathcal{N}(y, 1) \qquad z \sim q_\phi(z \mid x)$$

$$\frac{dz}{dy} = 2y \qquad \frac{dz}{dy} = ?? \qquad \frac{dz}{d\phi} = ??$$

Reparameterization Trick

$$z \sim \mathcal{N}(\mu, \sigma)$$

$$\frac{\partial z}{\partial \mu} = ??$$

$$\frac{\partial z}{\partial \sigma} = ??$$

$$z = \epsilon_i \times \sigma + \mu$$

$$\epsilon \sim \mathcal{N}(0, 1)$$

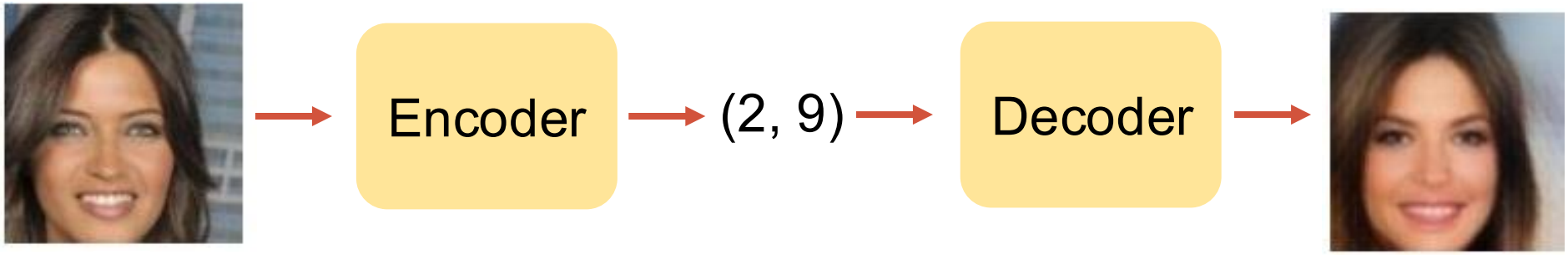
$$\frac{\partial z}{\partial \mu} = 1$$

$$\frac{\partial z}{\partial \sigma} = \epsilon_i$$

$$\min_{q_\phi(\mathbf{z}|\mathbf{x})} \mathrm{KL}[q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]$$

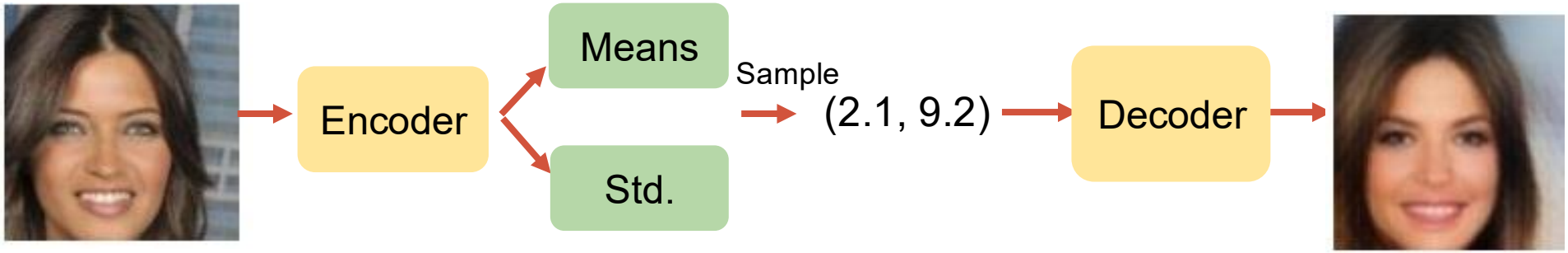
$$\Longleftrightarrow \quad \min_{q_\phi(\mathbf{z}|\mathbf{x})} \mathrm{KL}[q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})] - \mathbb{E}_{p(\epsilon)}[\log p_\theta(\mathbf{x} \mid \hat{\mathbf{z}})]$$

$$\hat{\mathbf{z}} = \epsilon \cdot \sigma + \mu, \quad p(\epsilon) \sim \mathcal{N}(0, I)$$



X X'

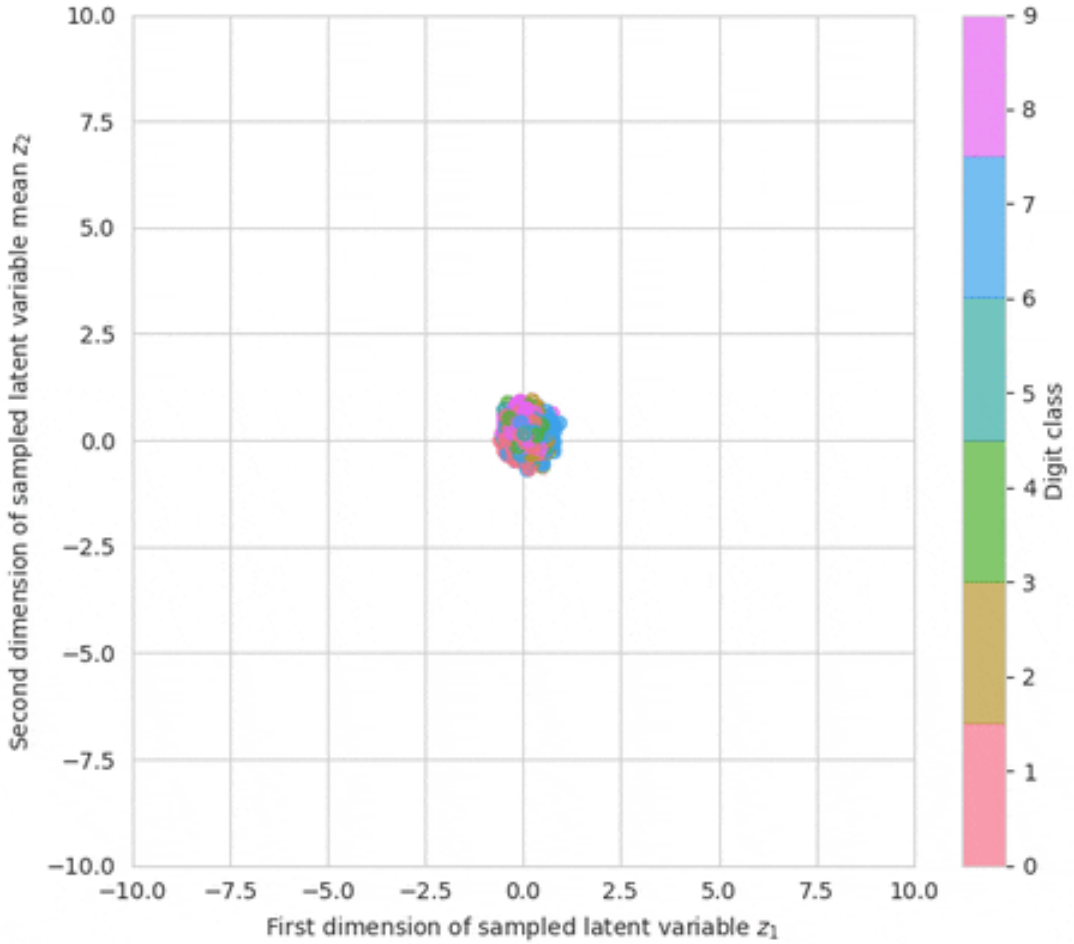
Standard AE Loss $\|X - X'\|^2$



X X'

Variational AE Loss $\approx \|X - X'\|^2 + \mathrm{KL}[q(\mathbf{z} \mid \mathbf{x}) \| \mathcal{N}(0, I)]$

More info: https://arxiv.org/abs/1606.05908



Notes: when we compresses data, meaningful “codes” tend to emerge.
The P(z)~N(0,1) clusters the data and allow for easy interpolation between images

https://jaan.io/what-is-variational-autoencoder-vae-tutorial/

VAE improvements

- Research focuses on disentangle meaning into different dimension of z (z_1 = hair color, z_2 = glasses, z_3 = age)
  - Controllable generation
- Improve quality of images

Disentangling Representation: β-VAE

We want compact meaningful representation.

But also disentangled representation

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta\, D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

https://openreview.net/forum?id=Sy2fzU9gl

Skin color

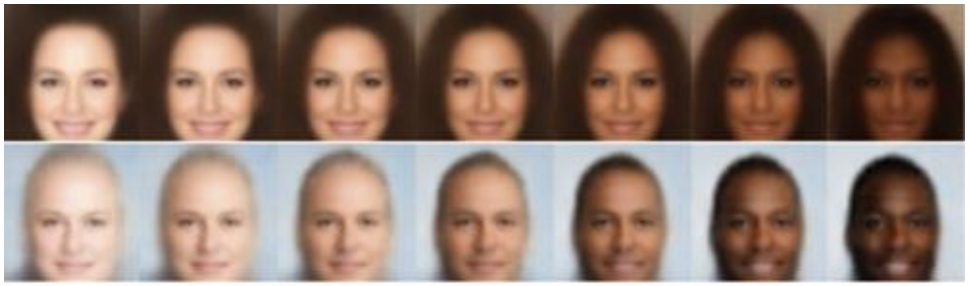
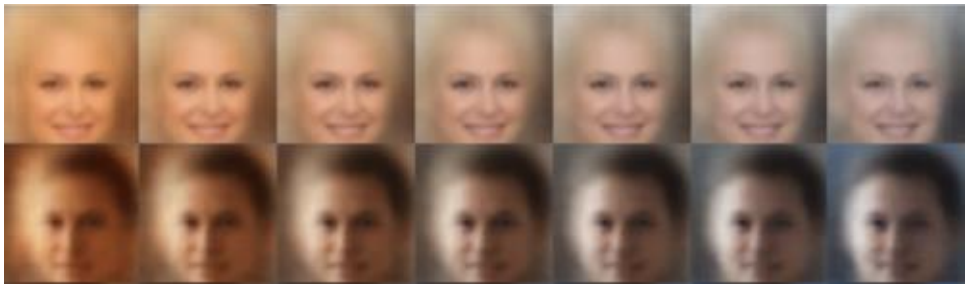


Image Saturation



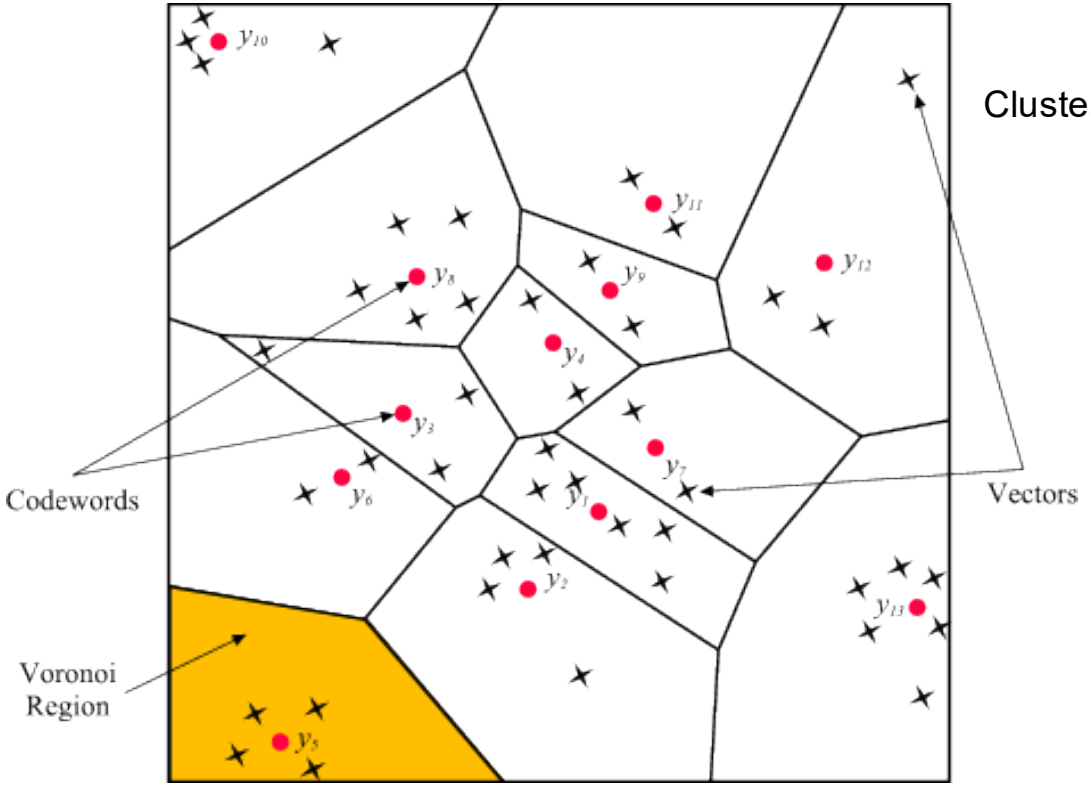
Head Angle

Age / Gender

VQ-VAE

https://arxiv.org/abs/1711.00937

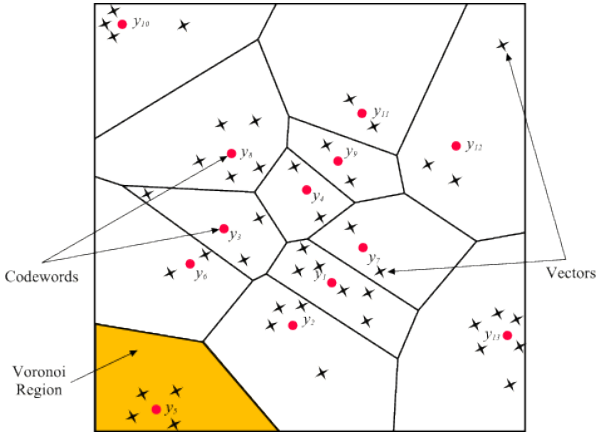
- Learn discrete z by using vector quantization



http://www.mqasem.net/vectorquantization/vq.html

VQ-VAE

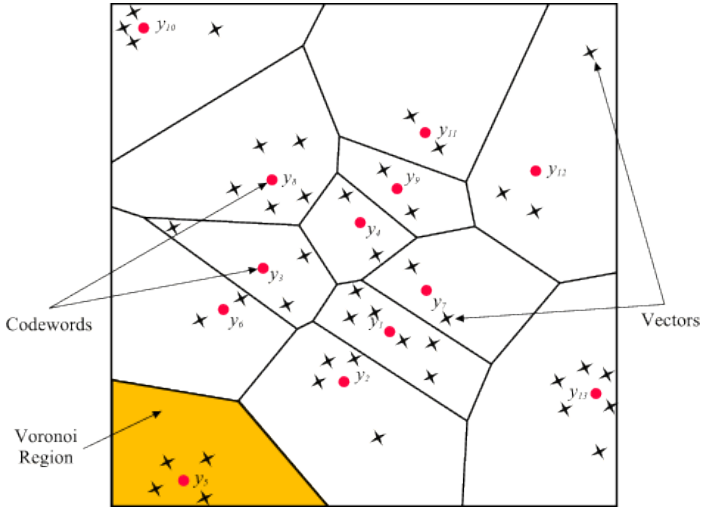
- An image can have multiple codes
- Encoder produces multiple vectors -> multiple codes
  - Ex. Image encodes into grid of NxN vector
- 32x32 grid vectors with 512 codes each
  - = 512^{32x32} possibilities



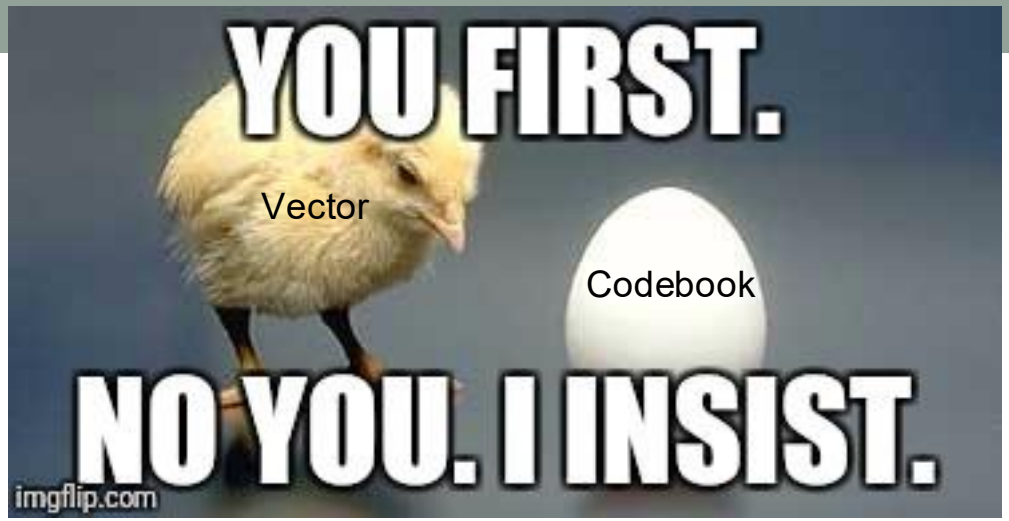
5 (girl), 10 (fire), 3 (road)

Learning the codebook

- Ideally, we want to have the encoded vectors be closed to the codewords
- And we want codewords that are good representations of encoded vectors



Overall loss



1.Codebook can reconstruct

2.Moves codebook to vector

3.Moves vector to codebook

Encoded vector

Code book

Hyperparameter to balance the loss

$$L = \log p(x|z_q(x)) + \|\mathrm{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \mathrm{sg}[e]\|_2^2,$$

Normal reconstruction loss

Stop gradient. No gradient flow through this term
Pytorch .detach() see homework ☺

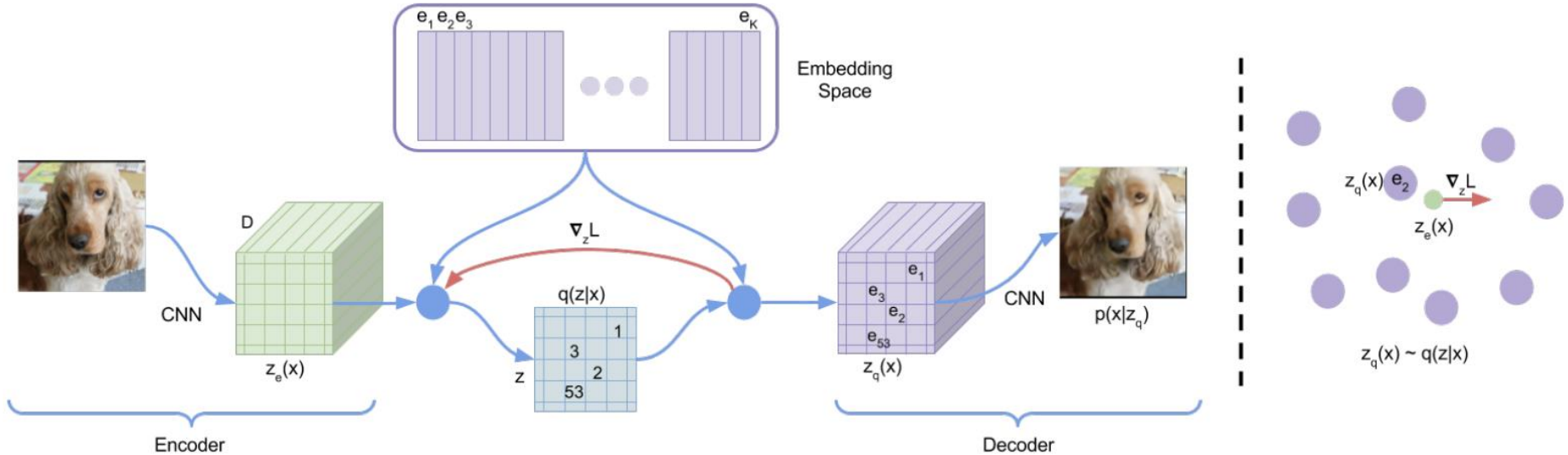


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2. The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

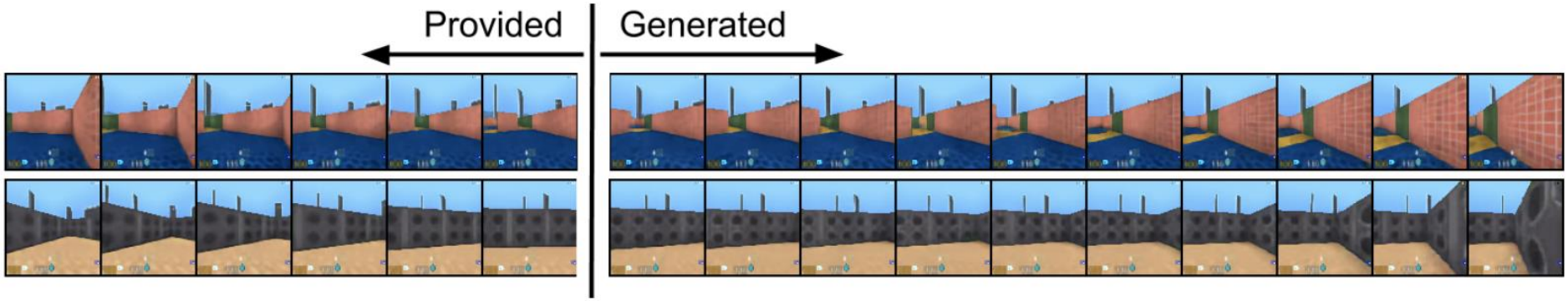
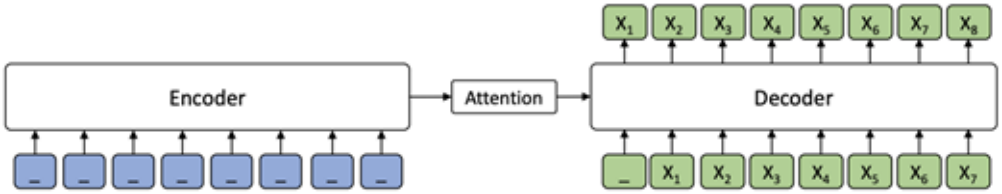


Figure 7: First 6 frames are provided to the model, following frames are generated conditioned on an action. Top: repeated action "move forward", bottom: repeated action "move right".

Dall E



Transformer (GPT3) + VQ VAE

More explanation here
https://ml.berkeley.edu/blog/posts/dalle2/

(a) a tapir made of accordion. a tapir with the texture of an accordion.

(b) an illustration of a baby hedgehog in a christmas sweater walking a dog

(c) a neon sign that reads "backprop". a neon sign that reads "backprop". backprop neon sign

(d) the exact same cat on the top as a sketch on the bottom

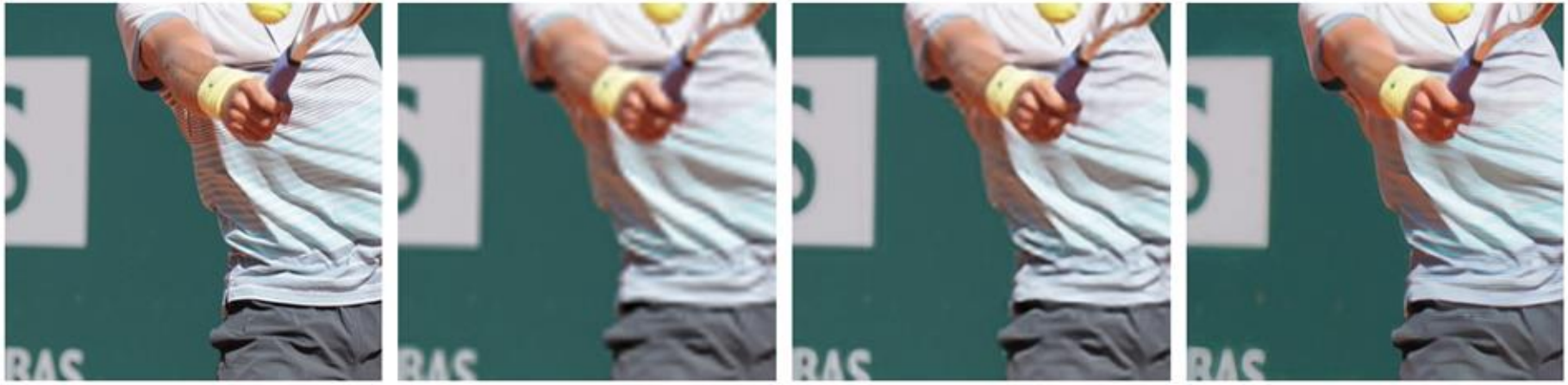
Figure 2. With varying degrees of reliability, our model appears to be able to combine distinct concepts in plausible ways, create anthropomorphized versions of animals, render text, and perform some types of image-to-image translation.

https://openai.com/blog/dall-e/ https://arxiv.org/pdf/2102.12092.pdf

Perceptual loss

Typical Autoencoder uses per pixel loss (for example L2)

Might not capture the important content



Ground Truth | Bicubic | SRCNN [11] | Perceptual loss

https://arxiv.org/abs/1603.08155

Perceptual loss

Measure semantic differences

Use the feature extracted from a pre-trained object classifier as a representation for semantic

Use the L2 between features as an additional loss

$$\ell_{feat}^{\phi,j}(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{y}) - \phi_j(y)\|_2^2$$

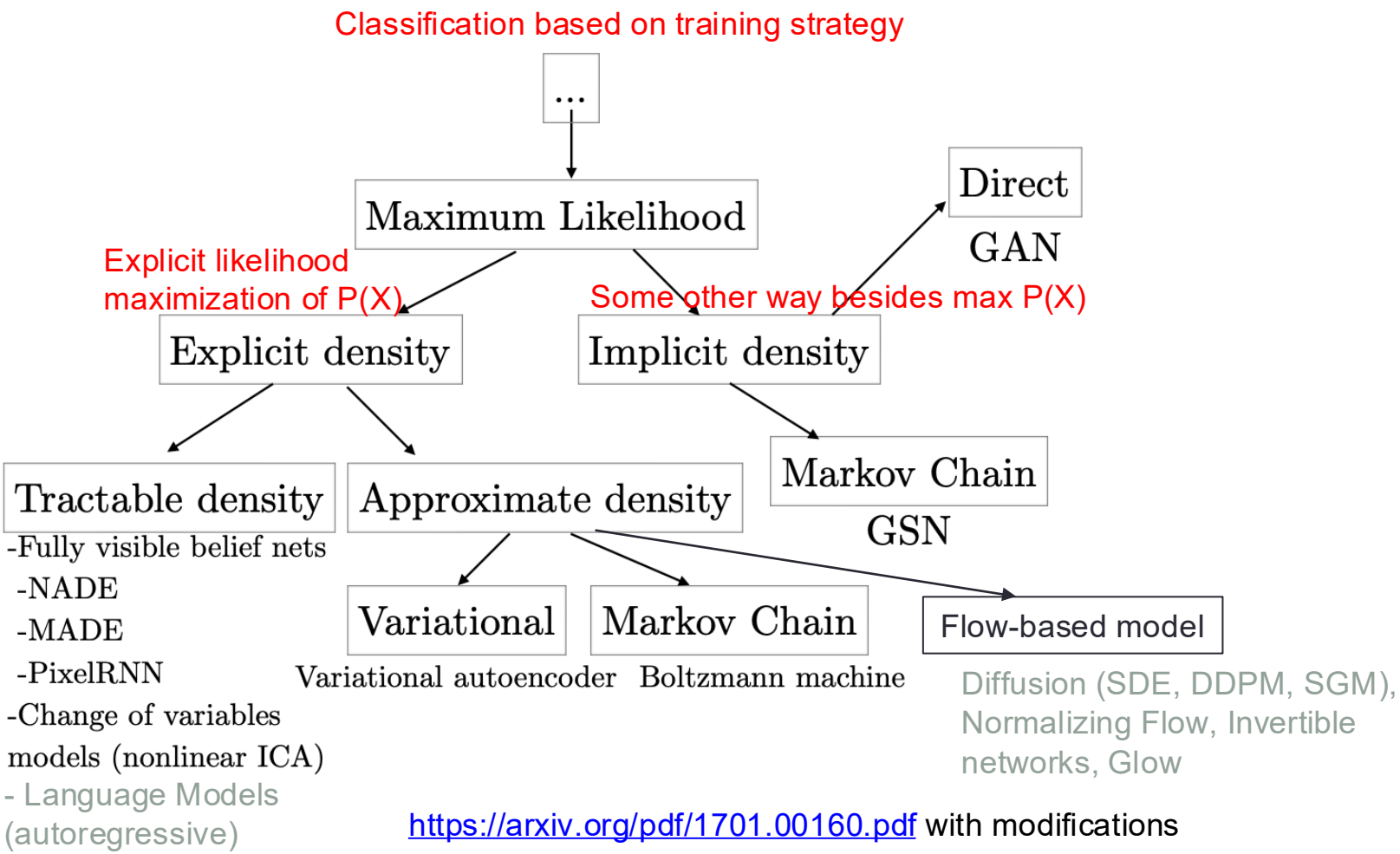
https://arxiv.org/abs/1603.08155

Deep generative models

- Autoencoders
  - Variational autoencoders
- GAN
- Diffusion/Flow

Implicit generative models

- We can generate without explicitly modeling P(X)



https://arxiv.org/pdf/1701.00160.pdf with modifications

Generating Images

VAE can be used to generate images by feeding numbers into the decoder.

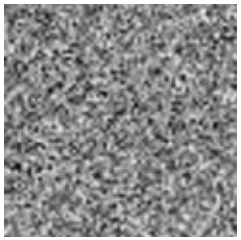
Are there other ways?

Given a collection of, say, faces, how can we generate new faces?

Maybe let’s start with a decoder, because we need something that outputs an image anyway.

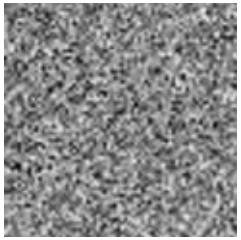
Generating Realistic Faces

Random Generator N(0, I) → (0.1, -0.5, ...) → Decoder →



What kind of loss should we use?

Generating Realistic Faces

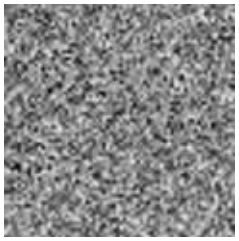
Random Generator N(0, I) → Decoder →  → f(**X**)

X

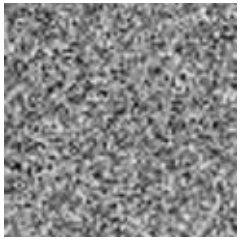
We'll be done if we can come up with a smooth function f, that takes in the output image x and outputs [0, 1] where:

f( ) = 1

f( ) = 0.2

f() = 0

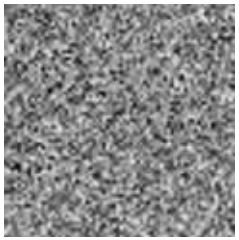
Generating Realistic Faces

Random Generator N(0, I) → Decoder →  → f(**X**)

X

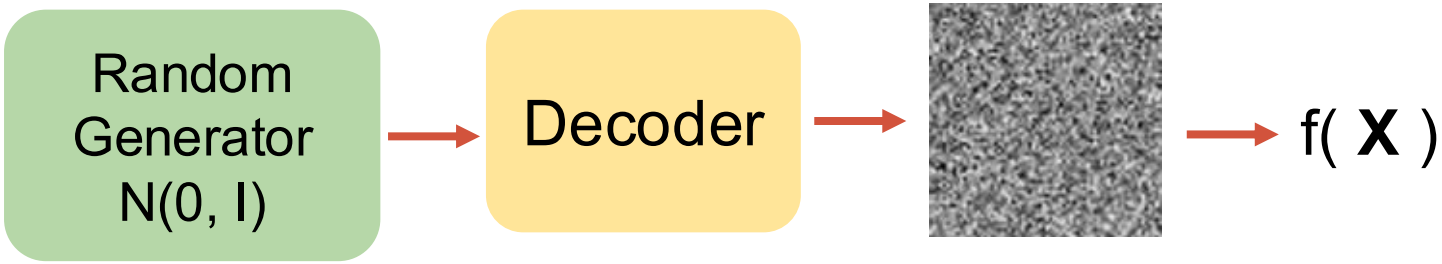
f( ) = 1

f( ) = 0.2

f() = 0

Given this f, we can compute its gradient with respect to the decoder parameters and use SGD to train. Done!

Generating Realistic Faces

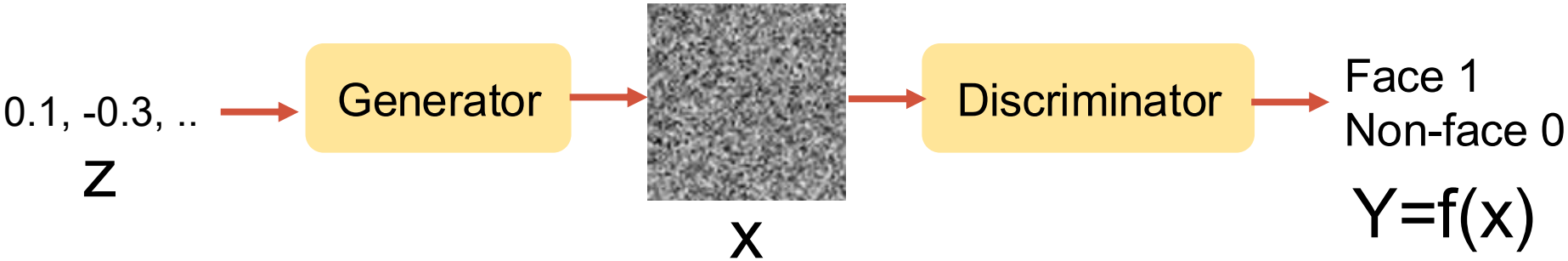


Solution: We will use another network as f. This network will predict the probability of x being a face.

Training this f is a simple supervised learning!

All face photos -> positive examples
All non-face photos -> negative examples

Generating Realistic Faces



Generative Adversarial Networks (GAN)

- Consider a money counterfeiter
  - He wants to make fake money that looks real
  - There's a police that tries to differentiate fake and real money.

- The counterfeiter is the adversary and is generating fake inputs. – Generator network

- The police tries to discriminate between fake and real inputs. – Discriminator network

Generative Adversarial Networks (GAN) – minimax loss

0.1, -0.3, ..
z

Generator

x

Discriminator

Real or Fake
1 0
Y=f(x)

- Generator (Money Faker):
  - Maximize Y

$$\min_G \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z}))]$$

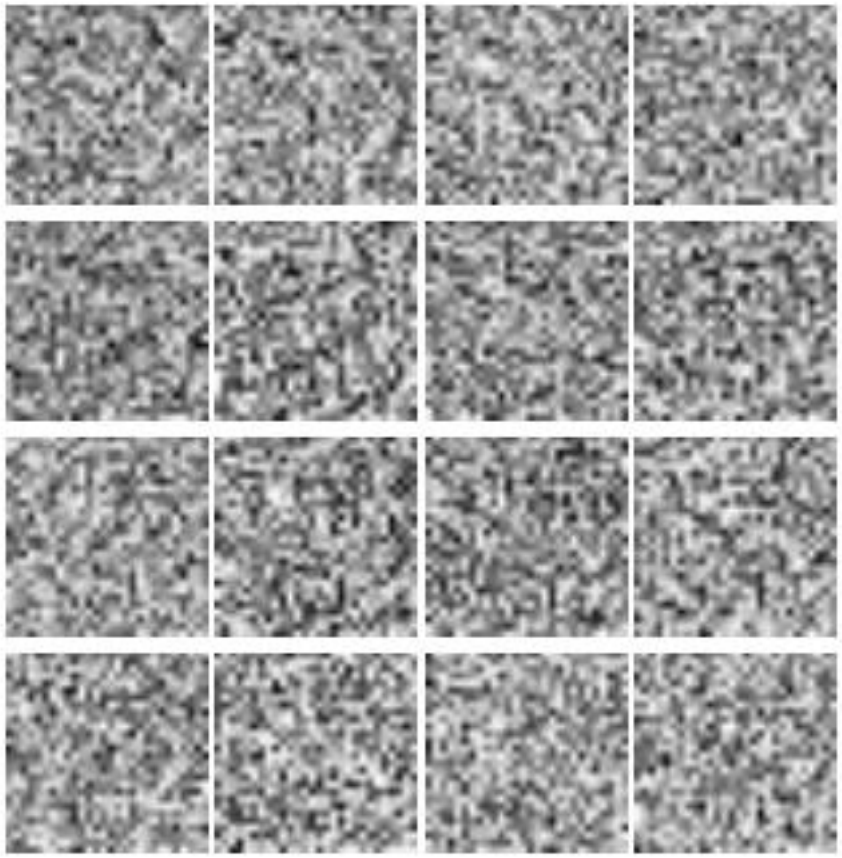
- Discriminator (Police):
  - For real images => Maximize Y
  - For generated images from the faker => Minimize Y

$$\max_D \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z}))] + \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log(D(\mathbf{x}))]$$

This comes from binary cross entropy loss

GAN example

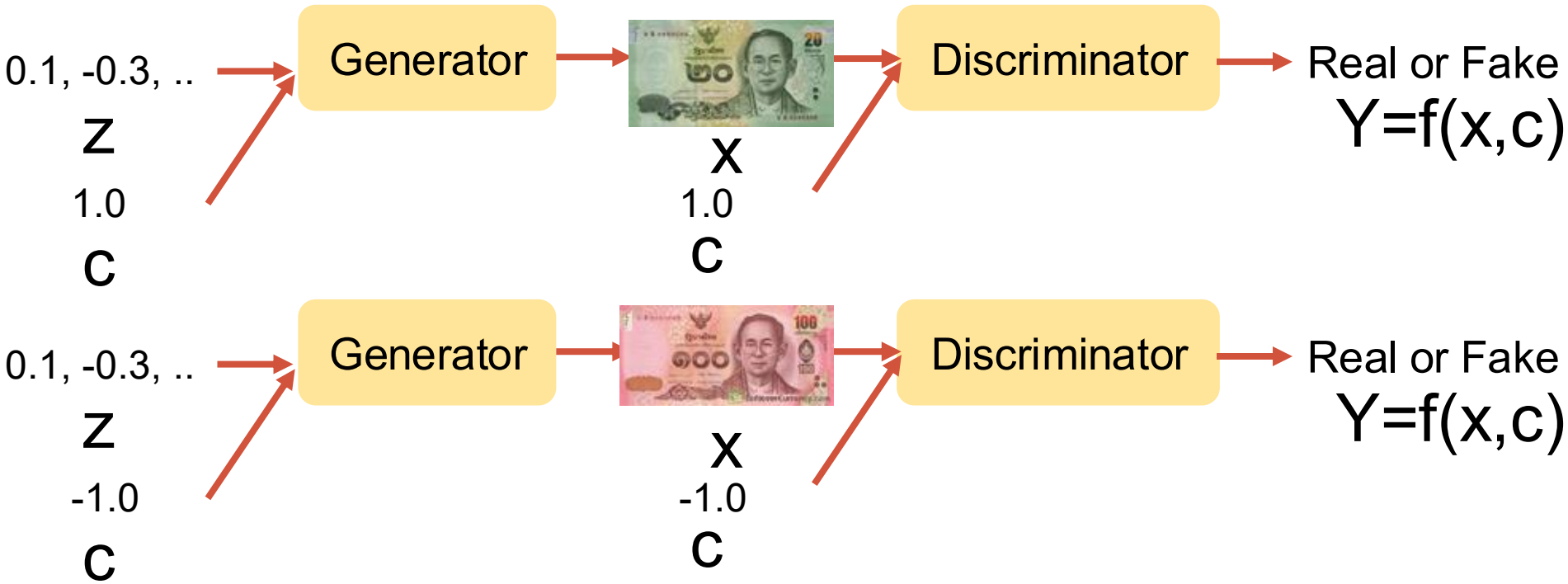
Generator output starts from random noise and gets better as we train.



https://wiseodd.github.io/techblog/2016/09/17/gan-tensorflow/

Conditional GAN

- Can conditioned GAN to generate a certain trait/class



Just concatenate C to the noise and the discriminator input
Discriminator also checks if it's the correct class

Mode collapse

Model only learn a couple types (modes) of inputs

https://arxiv.org/pdf/1701.07875.pdf

Ian Goodfellow
@goodfellow_ian

4.5 years of GAN progress on face generation.
arxiv.org/abs/1406.2661 arxiv.org/abs/1511.06434
arxiv.org/abs/1606.07536 arxiv.org/abs/1710.10196
arxiv.org/abs/1812.04948

2014

2015

2016

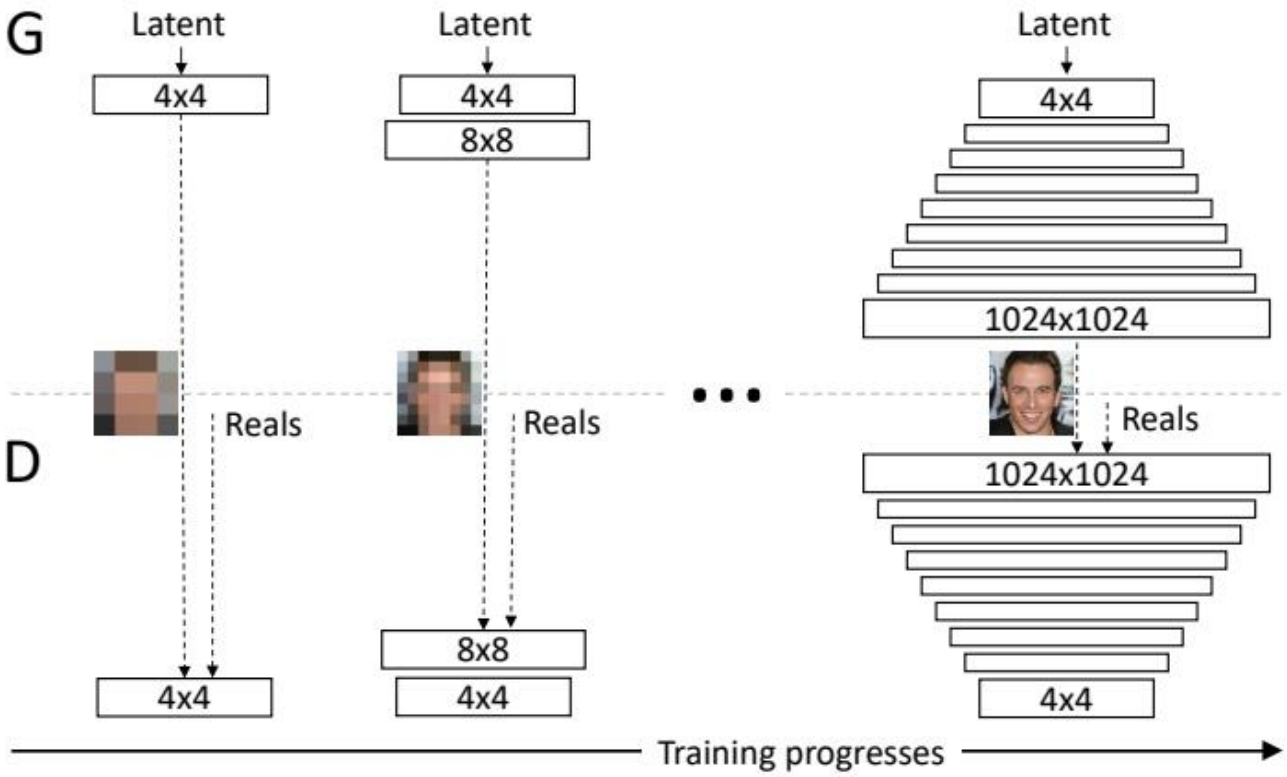
2017
Progressive GAN

2018
Style GAN

More realistic, higher res, more controllable

Progressive GANS

Newer GANs are usually trained from low res to high res

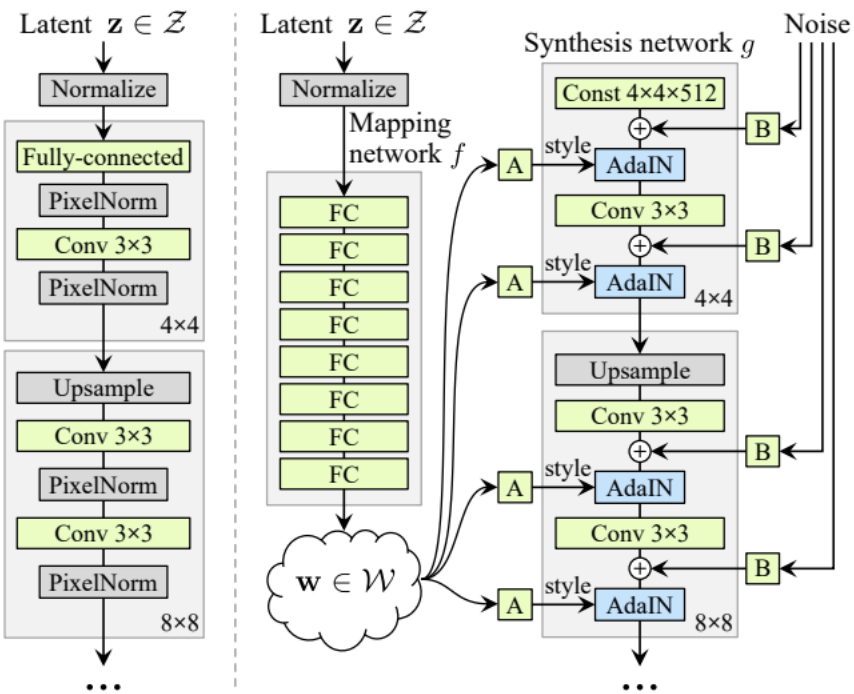


Progressive Growing of GANs for Improved Quality, Stability, and Variation [Karras et al. 2017]

https://drive.google.com/drive/folders/1I9wsMzDkxF85pZ39RTVjiP83BLIEXUBQ

Style-GAN (V1)

- Based on progressive GAN
- Inspired from style transfer network (no actual style transfer here)
- Embedded “Styles” from z
- Use styles to control the generator



(a) Traditional (b) Style-based generator

Instance norm

$$\mathrm{AdaIN}(\mathbf{x}_i, \mathbf{y}) = \mathbf{y}_{s,i} \frac{\mathbf{x}_i - \mu(\mathbf{x}_i)}{\sigma(\mathbf{x}_i)} + \mathbf{y}_{b,i}, \qquad (1)$$

Style value

V1 https://arxiv.org/abs/1812.04948 V2 https://arxiv.org/abs/1912.04958 V3 https://arxiv.org/abs/2106.12423

https://medium.com/nybles/understanding-machine-learning-through-memes-4580b67527bf

Extra reading
https://machinelearningmastery.com/generative-adversarial-network-loss-functions/

GANs Loss Formulations

$$\max_D \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z}))] + \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log(D(\mathbf{x}))]$$

$$\min_G \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z}))]$$

Discriminator | Generator

GAN	Discriminator Loss	Generator Loss
MM GAN	$\mathcal{L}_D^{GAN} = -\mathbb{E}_{x\sim p_d}[\log(D(x))] - \mathbb{E}_{\hat{x}\sim p_g}[\log(1 - D(\hat{x}))]$	$\mathcal{L}_G^{GAN} = \mathbb{E}_{\hat{x}\sim p_g}[\log(1 - D(\hat{x}))]$
NS GAN	$\mathcal{L}_D^{NSGAN} = -\mathbb{E}_{x\sim p_d}[\log(D(x))] - \mathbb{E}_{\hat{x}\sim p_g}[\log(1 - D(\hat{x}))]$	$\mathcal{L}_G^{NSGAN} = -\mathbb{E}_{\hat{x}\sim p_g}[\log(D(\hat{x}))]$
* WGAN	$\mathcal{L}_D^{WGAN} = -\mathbb{E}_{x\sim p_d}[D(x)] + \mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})]$	$\mathcal{L}_G^{WGAN} = -\mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})]$
WGAN GP	$\mathcal{L}_D^{WGANGP} = \mathcal{L}_D^{WGAN} + \lambda\mathbb{E}_{\hat{x}\sim p_g}[(\|\|\nabla D(\alpha x + (1 - \alpha\hat{x})\|\|_2 - 1)^2]$	$\mathcal{L}_G^{WGANGP} = -\mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})]$
*LS GAN	$\mathcal{L}_D^{LSGAN} = -\mathbb{E}_{x\sim p_d}[(D(x) - 1)^2] + \mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})^2]$	$\mathcal{L}_G^{LSGAN} = -\mathbb{E}_{\hat{x}\sim p_g}[(D(\hat{x} - 1))^2]$
*DRAGAN	$\mathcal{L}_D^{DRAGAN} = \mathcal{L}_D^{GAN} + \lambda\mathbb{E}_{\hat{x}\sim p_d+\mathcal{N}(0,c)}[(\|\|\nabla D(\hat{x})\|\|_2 - 1)^2]$	$\mathcal{L}_G^{DRAGAN} = \mathbb{E}_{\hat{x}\sim p_g}[\log(1 - D(\hat{x}))]$
BEGAN	$\mathcal{L}_D^{BEGAN} = \mathbb{E}_{x\sim p_d}[\|\|x - \mathrm{AE}(x)\|\|_1] - k_t\mathbb{E}_{\hat{x}\sim p_g}[\|\|\hat{x} - \mathrm{AE}(\hat{x})\|\|_1]$	$\mathcal{L}_G^{BEGAN} = \mathbb{E}_{\hat{x}\sim p_g}[\|\|\hat{x} - \mathrm{AE}(\hat{x})\|\|_1]$

Are GANs Created Equal? A Large-Scale Study [Lucic et al. 2018]

Wasserstein GAN (WGAN)

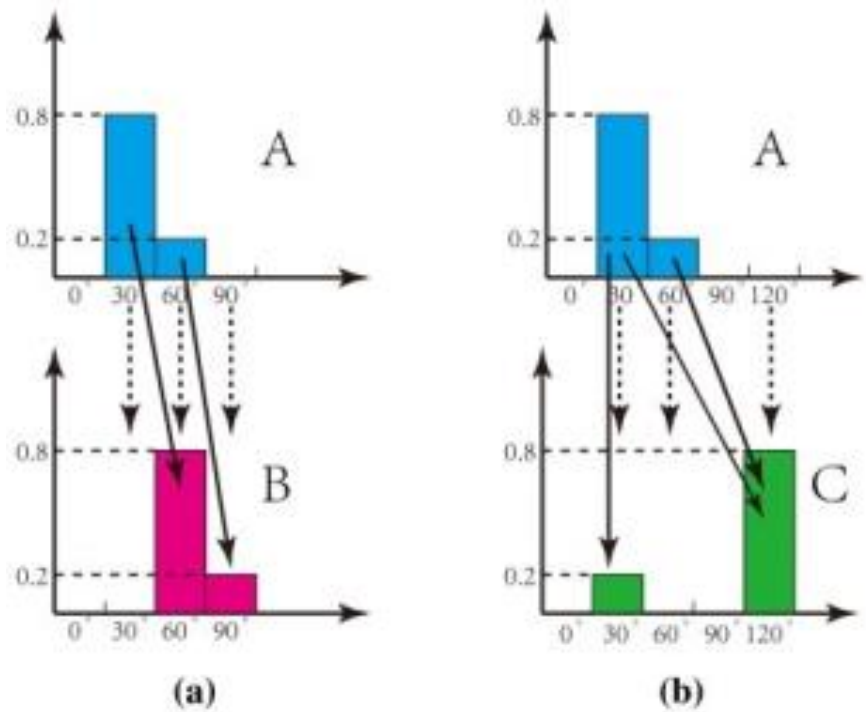
Wasserstein distance? (Earth mover distance)

Energy required to move mass to make two distributions look the same

WGAN

approximates and minimizes this distance

(Note vanilla GAN minimizes Jensen-Shannon divergence)



https://medium.com/@adriensieg/text-similarities-da019229c894

https://arxiv.org/abs/1701.07875
https://arxiv.org/abs/1704.00028

WGAN

Discriminator as a critic (no fake/real sigmoid) but output a score (linear activation at output layer)

WD has better gradient and convergence

	Discriminator/Critic	Generator
GAN	$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right]$	$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)$
WGAN	$\nabla_{w} \frac{1}{m} \sum_{i=1}^{m} \left[f\left(x^{(i)}\right) - f\left(G\left(z^{(i)}\right)\right)\right]$	$\nabla_{\theta} \frac{1}{m} \sum_{i=1}^{m} f\left(G\left(z^{(i)}\right)\right)$

WGAN

Approximating EMD requires the critic model to be a k-Lipschitz function

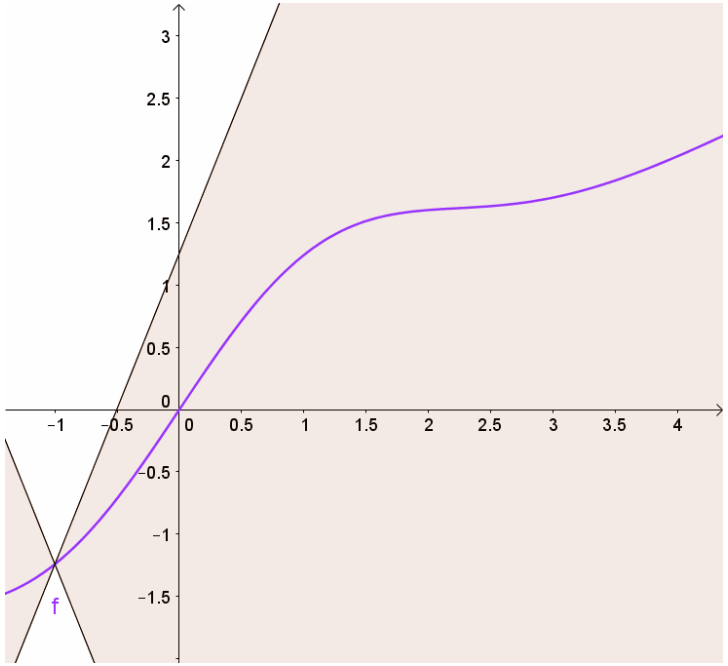
k-Lipschitz?

Bounded in slope of k

$|f(a) - f(b)| < k|a - b|$

Example

$f(x) = 5x$ is 5-Lipschitz



https://en.wikipedia.org/wiki/Lipschitz_continuity

WGAN to WGAN-GP

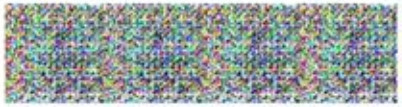
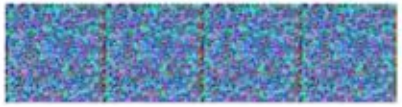
To make k-Lipschitz

WGAN caps the weights of all layers to 1

WGAN-GP improves and add Gradient Penalty to reduce the weights instead

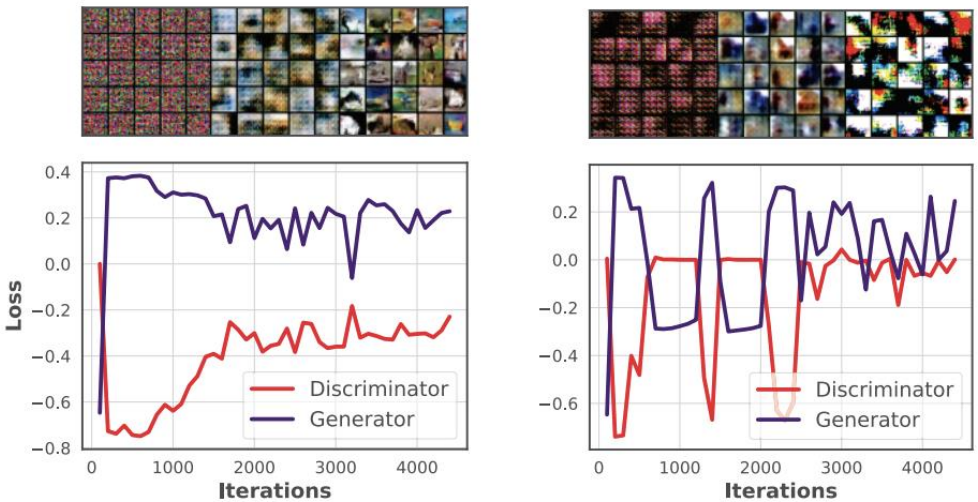
A differentiable function f is 1-Lipschtiz if and only if it has gradients with norm at most 1 everywhere.

$$L = \underbrace{\mathop{\mathbb{E}}_{\tilde{\boldsymbol{x}} \sim \mathbb{P}_g}\left[D(\tilde{\boldsymbol{x}})\right] - \mathop{\mathbb{E}}_{\boldsymbol{x} \sim \mathbb{P}_r}\left[D(\boldsymbol{x})\right]}_{\text{Original critic loss}} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{\boldsymbol{x}} \sim \mathbb{P}_{\hat{\boldsymbol{x}}}}\left[\left(\|\nabla_{\hat{\boldsymbol{x}}} D(\hat{\boldsymbol{x}})\|_2 - 1\right)^2\right]}_{\text{Our gradient penalty}}.$$

DCGAN	LSGAN	WGAN (clipping)	WGAN-GP (ours)
Baseline (G: DCGAN, D: DCGAN)			
G: No BN and a constant number of filters, D: DCGAN			
			
G: 4-layer 512-dim ReLU MLP, D: DCGAN			
No normalization in either G or D			
Gated multiplicative nonlinearities everywhere in G and D			
tanh nonlinearities everywhere in G and D			
101-layer ResNet G and D			
			

GAN without competition

- Minimax nature of GAN makes it hard to train
  - Optimize the duality gap instead



(a) Convergence (b) Divergence

Figure 1. Loss curves throughout the training progress of WGAN on the CIFAR-10 dataset.

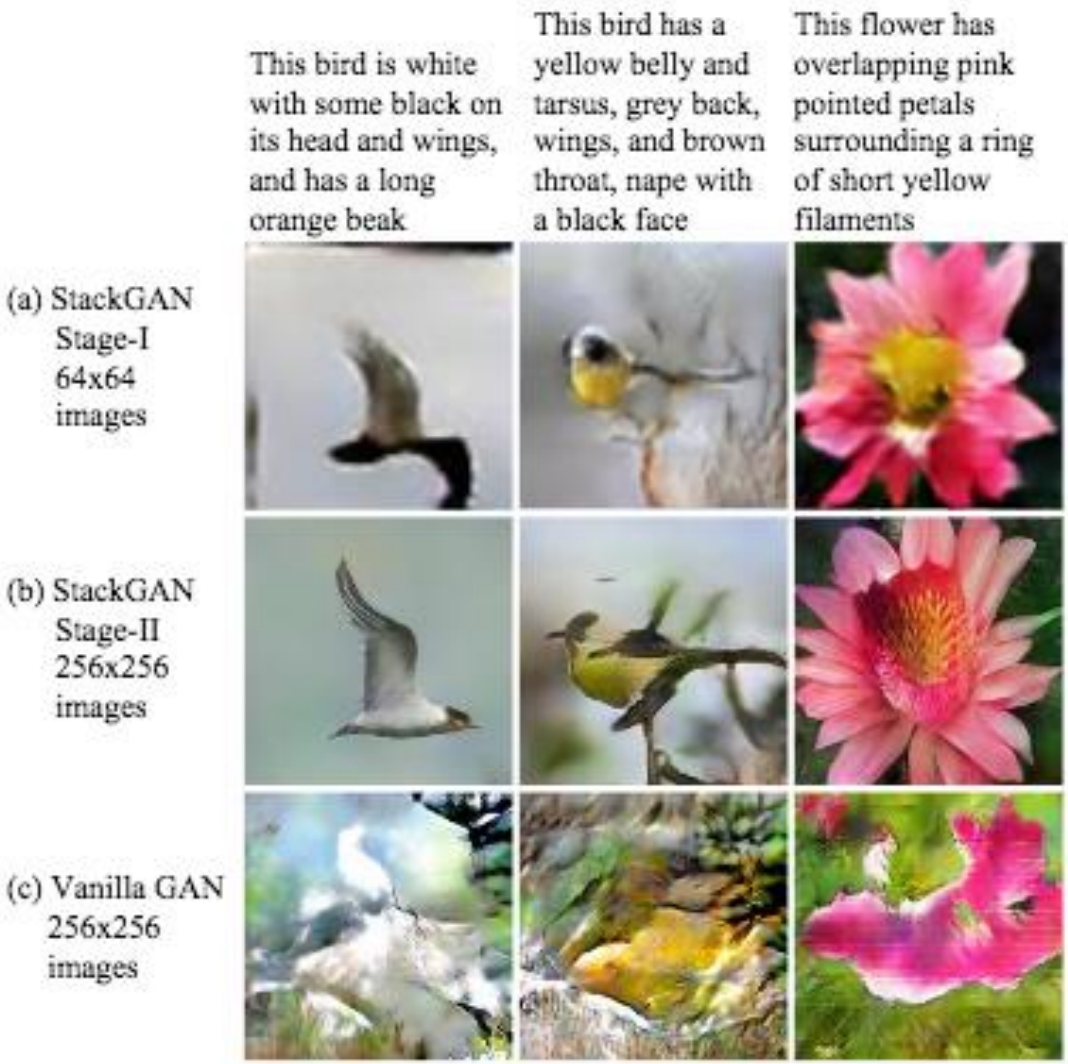
$$
\begin{aligned}
DG^{\lambda}(\theta_d, \theta_g) &= V_{D_w}(\theta_g) - V^{\lambda}_{G_w}(\theta_d), \textit{ where} \\
V_{D_w}(\theta_g) &= \max_{\theta'_d \in \Theta_D} V(D_{\theta'_d}, G_{\theta_g}) \\
V^{\lambda}_{G_w}(\theta_d) &= \min_{\theta'_g \in \Theta_G} V^{\lambda}(D_{\theta_d}, G_{\theta'_g})
\end{aligned}
$$

Estiamte V_D by optimizing discriminator to estimate worst case discriminator
Estiamte V_G by optimizing generator to estimiate worst case generator
Optimize the gap

Generative Minimization Networks: Training GANs Without Competition
https://arxiv.org/pdf/2103.12685.pdf
On Characterizing GAN Convergence Through Proximal Duality Gap
http://proceedings.mlr.press/v139/sidheekh21a/sidheekh21a.pdf

Other uses

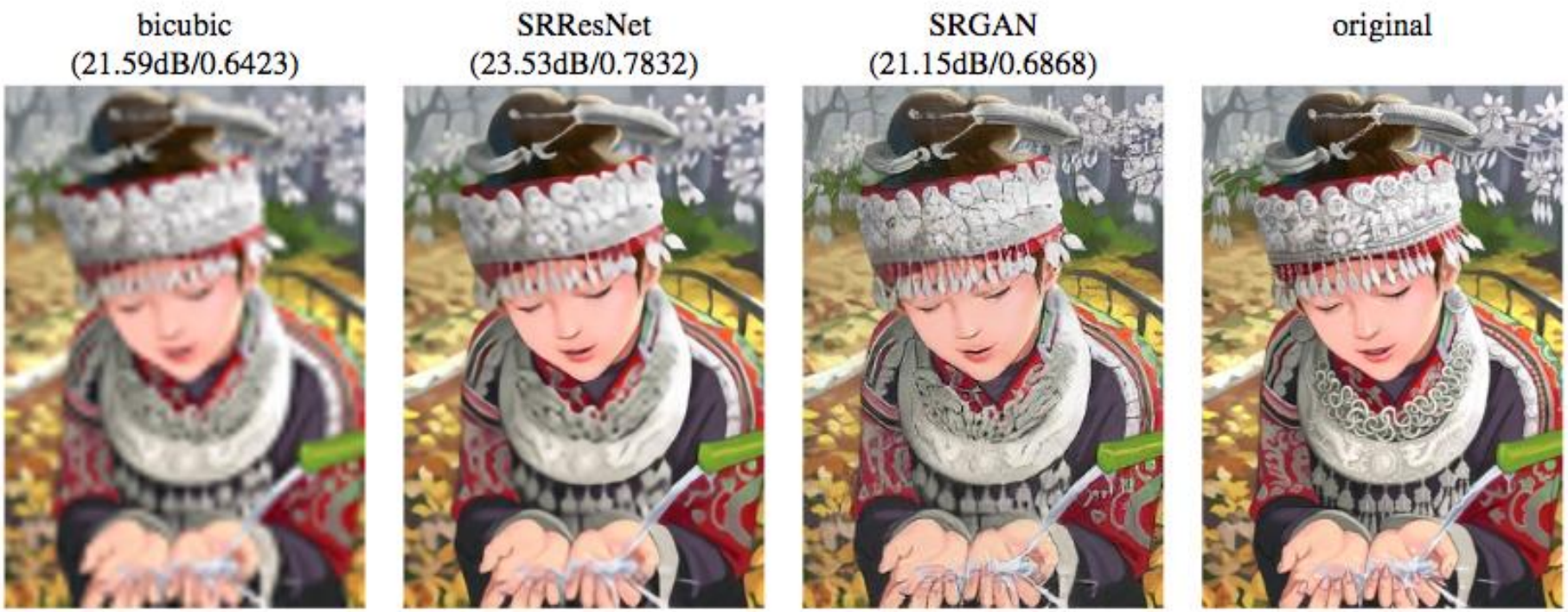
- StackGAN: text to photo



StackGAN: Text to Photo-realistic Image Synthesis with Stacked GANs [Zhang et al. 2017]

SRGAN

- https://arxiv.org/pdf/1609.04802.pdf

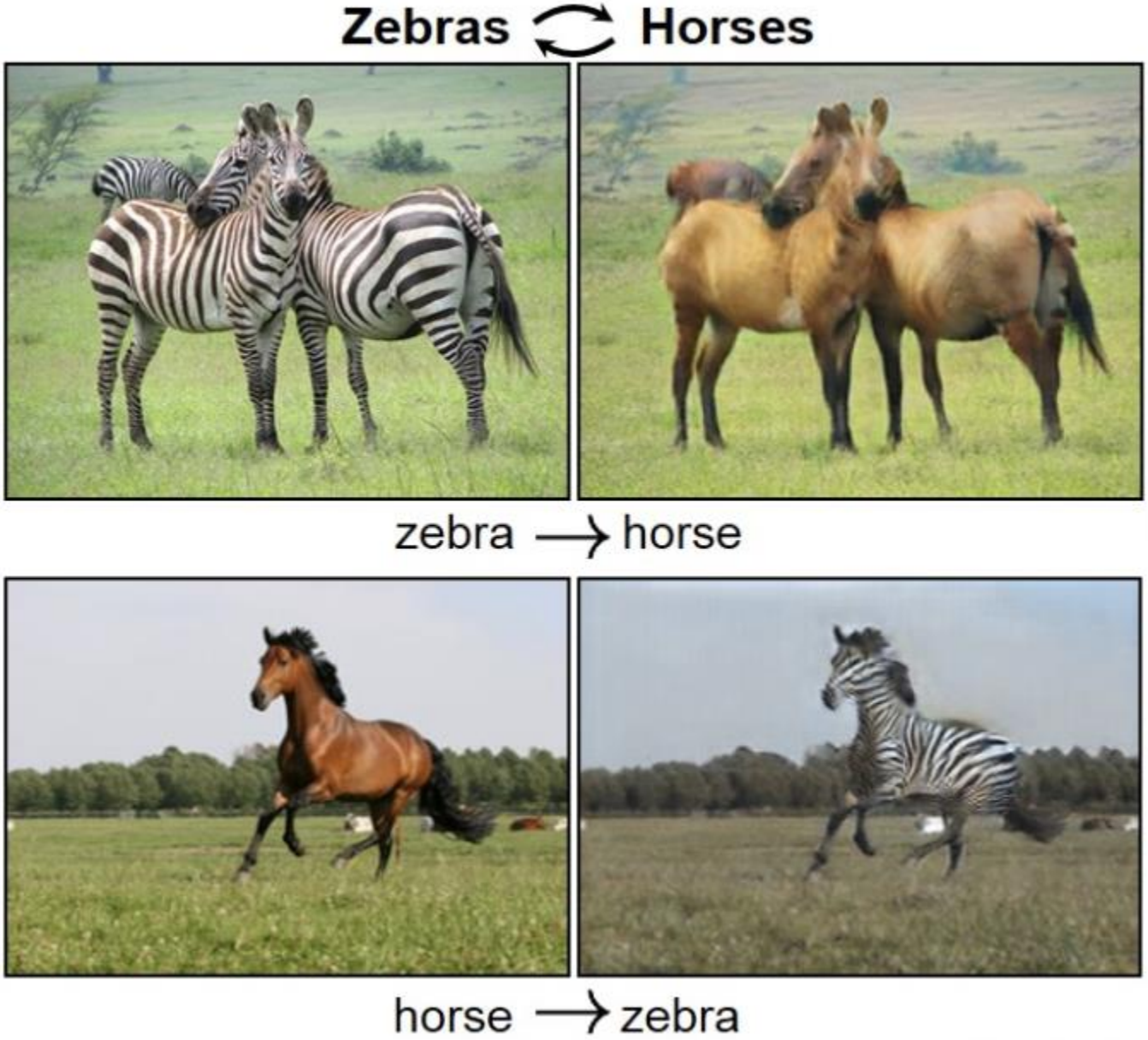


Content-aware Filling

Globally and Locally Consistent Image Completion [Iizuka et al.,

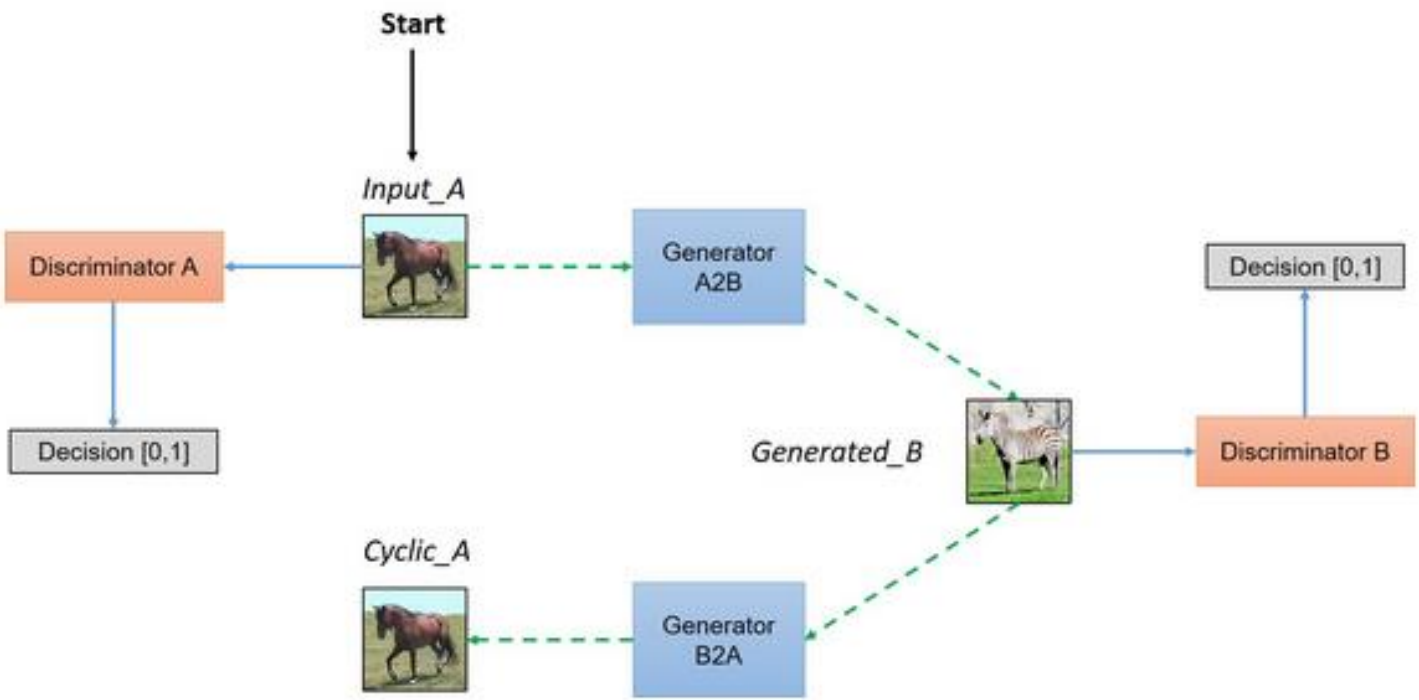
Style transfer with cycleGAN

https://hardikbansal.github.io/CycleGANBlog/



Cycle consistency

Note, you don’t need a paired dataset

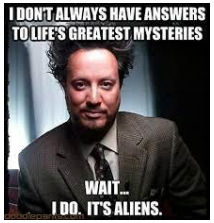


Cycle consistency
No need for parallel data!!

Adversarial debiasing

- Use the discriminator to force the encoder to drop bias information

Training data



Encoder

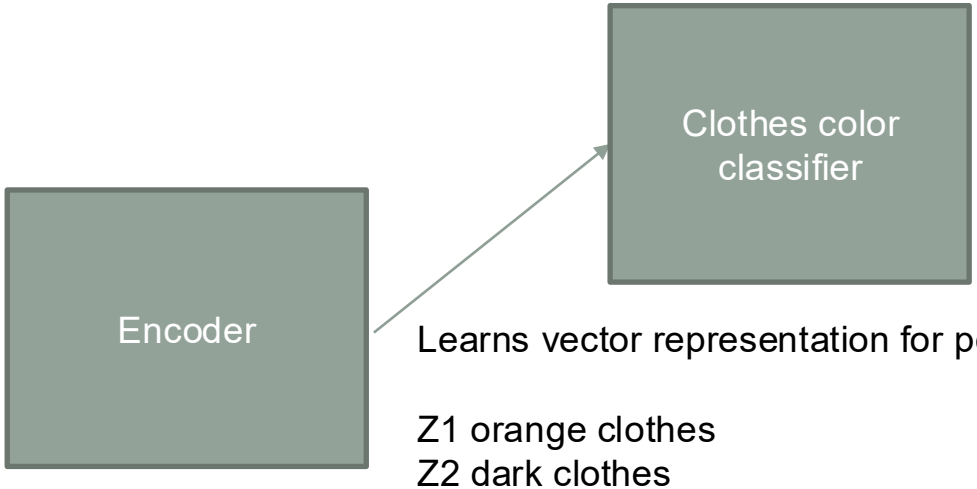
Learns vector representation for person ID

Z1 orange clothes
Z2 dark clothes
Z3 beard
Z4 hands up

How to remove unwanted information in latent code?

Adversarial debiasing

Want to have this classifier not be able to distinguish clothes color



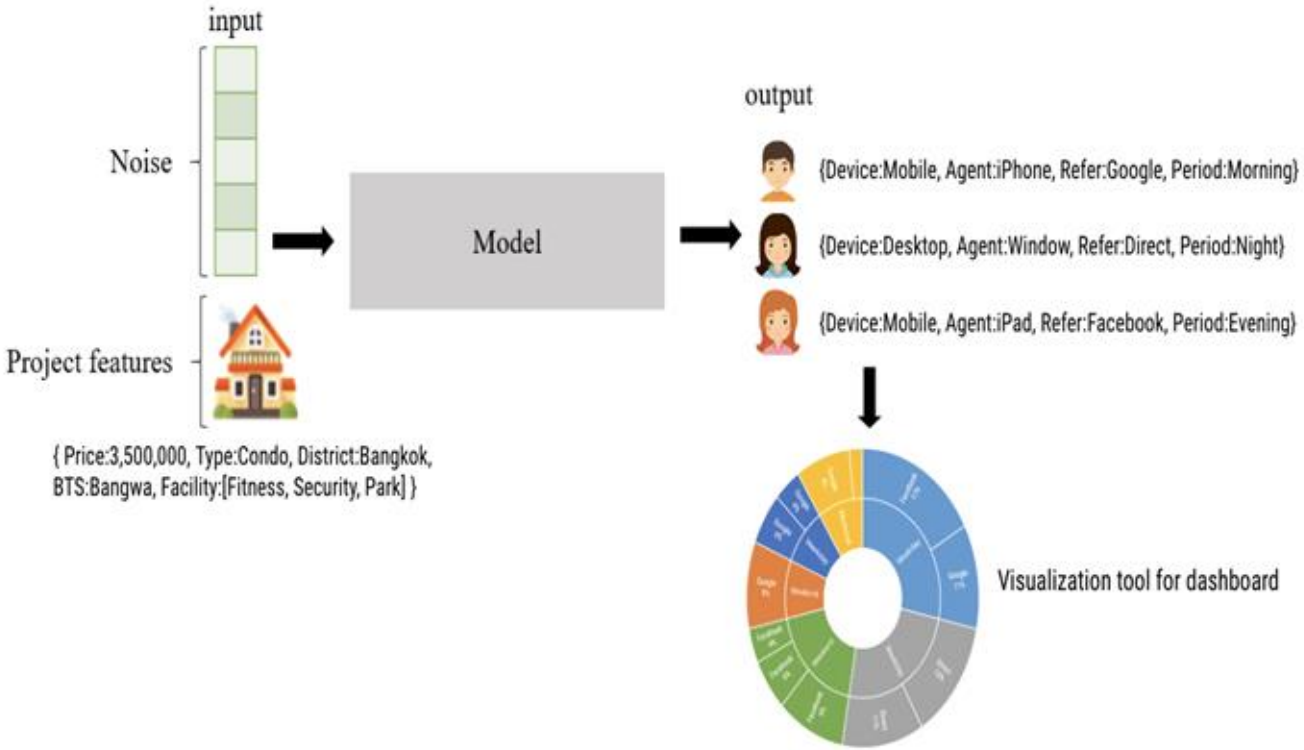
Max the cross entropy of this classifier to force the latent code to have no clothes information

Learns vector representation for person ID

Z1 orange clothes
Z2 dark clothes
Z3 beard
Z4 hands up

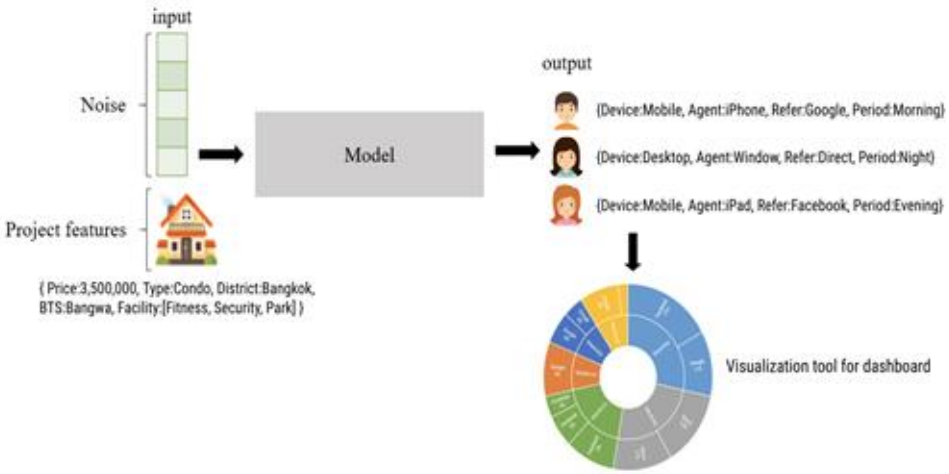
Reading https://arxiv.org/pdf/1801.07593.pdf
Example application usages
Bioinformatics https://www.biorxiv.org/content/10.1101/2021.04.14.439903v1.full.pdf
Face recognition https://arxiv.org/pdf/1911.08080.pdf

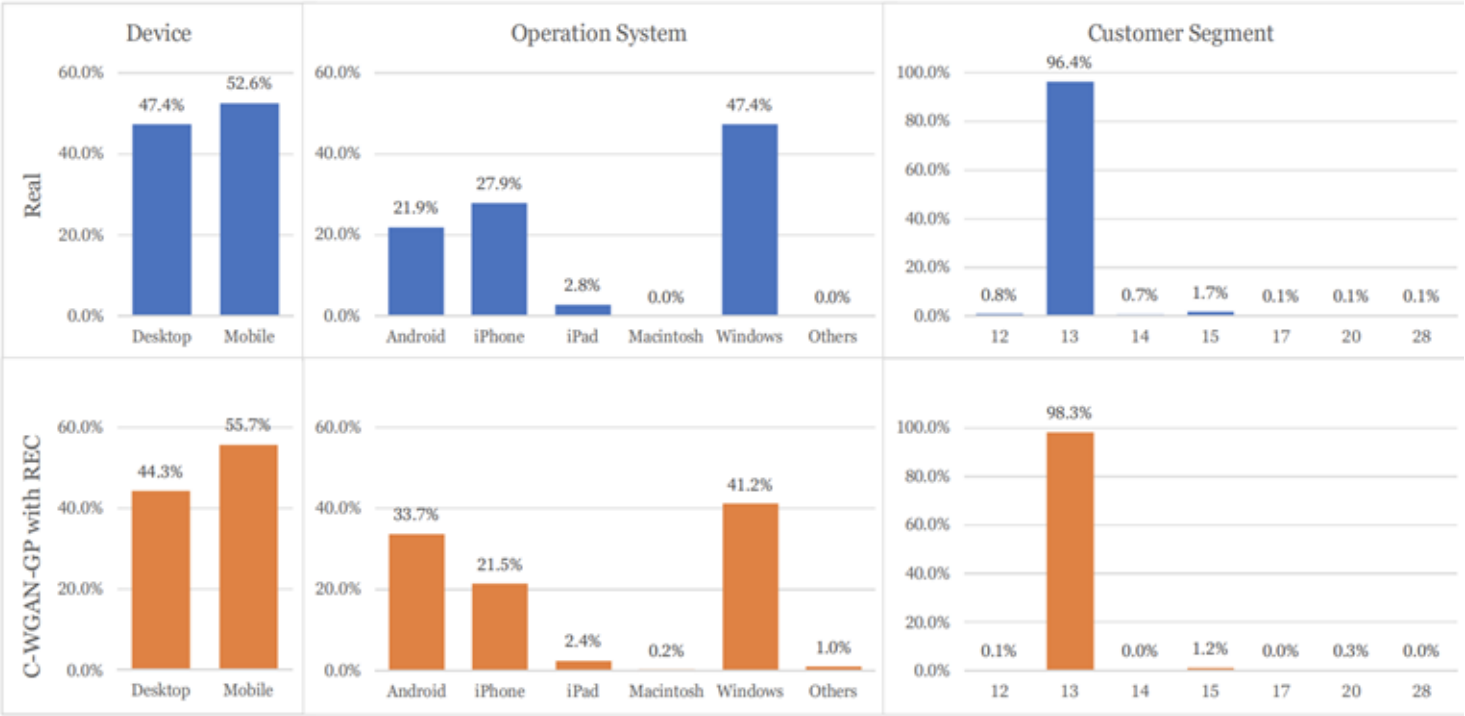
New product development



https://ieeexplore.ieee.org/document/9016238

Predicting users





https://ieeexplore.ieee.org/document/9016238

GAN vs VAE

- Typically GAN provides better quality but harder to train (mode collapse)
- Typically VAE provides more controllability
  - Representation more disentangled
  - Can easily interpolate two inputs
- VAE can encode nice embeddings/features for other task

- Nowsaday most methods are diffusion-based with some tricks/techniques from GAN/VAE

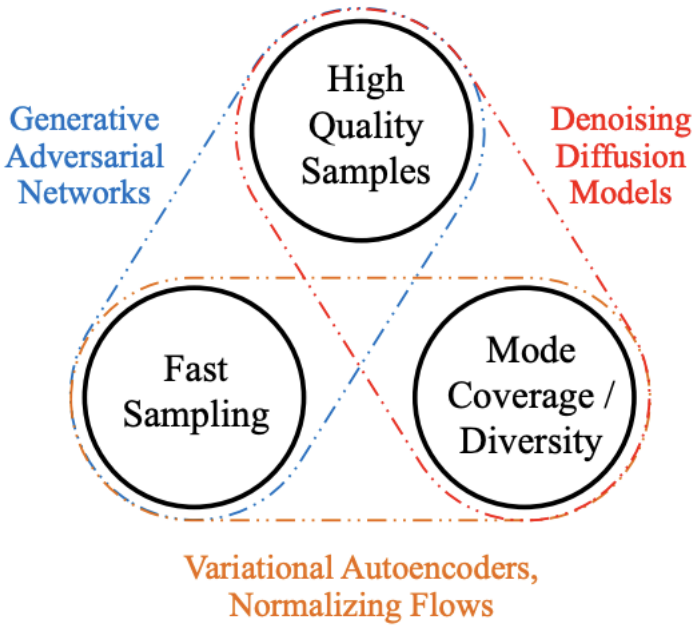


Figure 1: Generative learning trilemma.

https://arxiv.org/pdf/2112.07804